

NimblePros

TRAINING & MENTORING

ONE-DAY WORKSHOP

Software Architecture Fundamentals

Everything is a trade-off

Source: Mark Richards & Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020)

Before We Start

Wi-Fi

SSID: Hotel Utrecht Business

Password: ValkEvent

Parking

Grab a parking pass if you paid for parking

Student Repo

https://github.com/ardalis/software-architecture-fundamentals-workshop

https://bit.ly/3Wuz0yu

Morning

1. Welcome and Setup
2. Laws and Principles
3. "-ilities" and Trade-off Analysis
4. Monolithic Architecture Styles
5. Distributed Systems Fundamentals

Afternoon

6. SOA, DDD, and Microservices
7. Modularity vs. Granularity
8. Architecture Decision Records
9. Governance and Fitness Functions
10. Saga Patterns
11. Wrap-Up

01

SECTION

Welcome and Setup

Module 1 · 15 min

NimblePros
TRAINING & MENTORING

Why are you here?

What do you want to take away from today?



Photo: Bertrand Zuchuat / Unsplash · Unsplash License

Everything in software architecture is a trade-off

If you think you found one that isn’t, you haven’t identified the trade-off yet.

Source: Mark Richards & Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020)

NimblePros
TRAINING & MENTORING

Exercise 0: Set up your machine

- .NET 10 SDK, Docker running, and the Aspire CLI
- The workshop repo cloned, and a sample's tests passing
- Claude (or another AI agent) ready for the AI exercises

On your own · before the workshop, 5 min check now · exercises/setup.md



Photo: Galymzhan Abdugalimov / Unsplash · Unsplash License

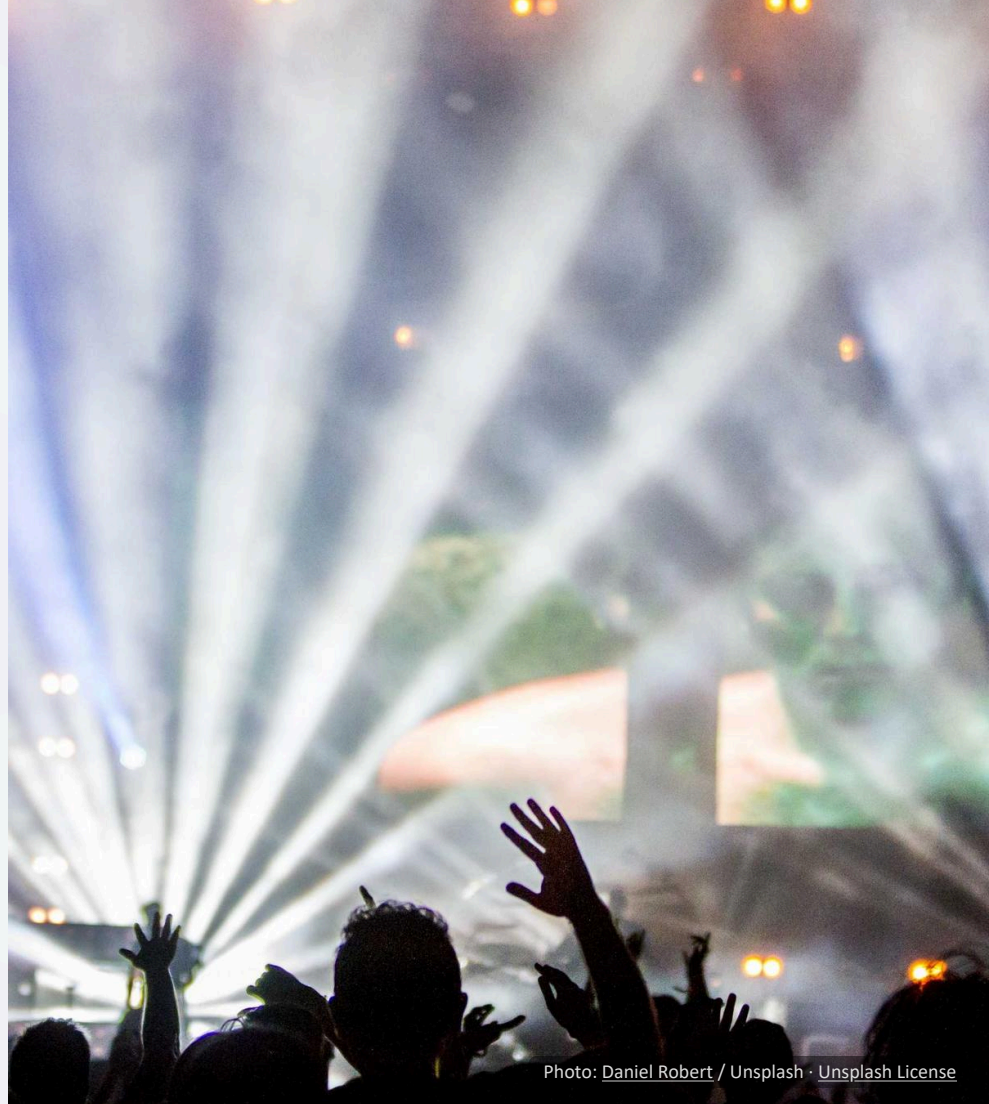
NimblePros
TRAINING & MENTORING

TicketTide

Meet TicketTide

Ticketing for about 120 independent venues, from comedy rooms to 18,000-seat arenas.

- **The case study for every exercise** today
- 40 employees, including 9 .NET developers in two teams
- Replacing a 12-year-old system that nobody wants to touch


Photo: Daniel Robert / Unsplash · Unsplash License

What TicketTide does

Who uses it

- **Fans** buy tickets on the web and in the app
- **Venue managers** publish events, seat maps, and prices
- **Door staff** scan tickets at the entrance
- **Support** handles refunds, transfers, and fraud

What it has to do

- Hold seats for 8 minutes during checkout
- Charge cards through an external payment provider
- Email QR-code tickets that fans can transfer
- Scan each ticket once, even with bad cell coverage
- Report sales to venues, and settle with them weekly

02

SECTION

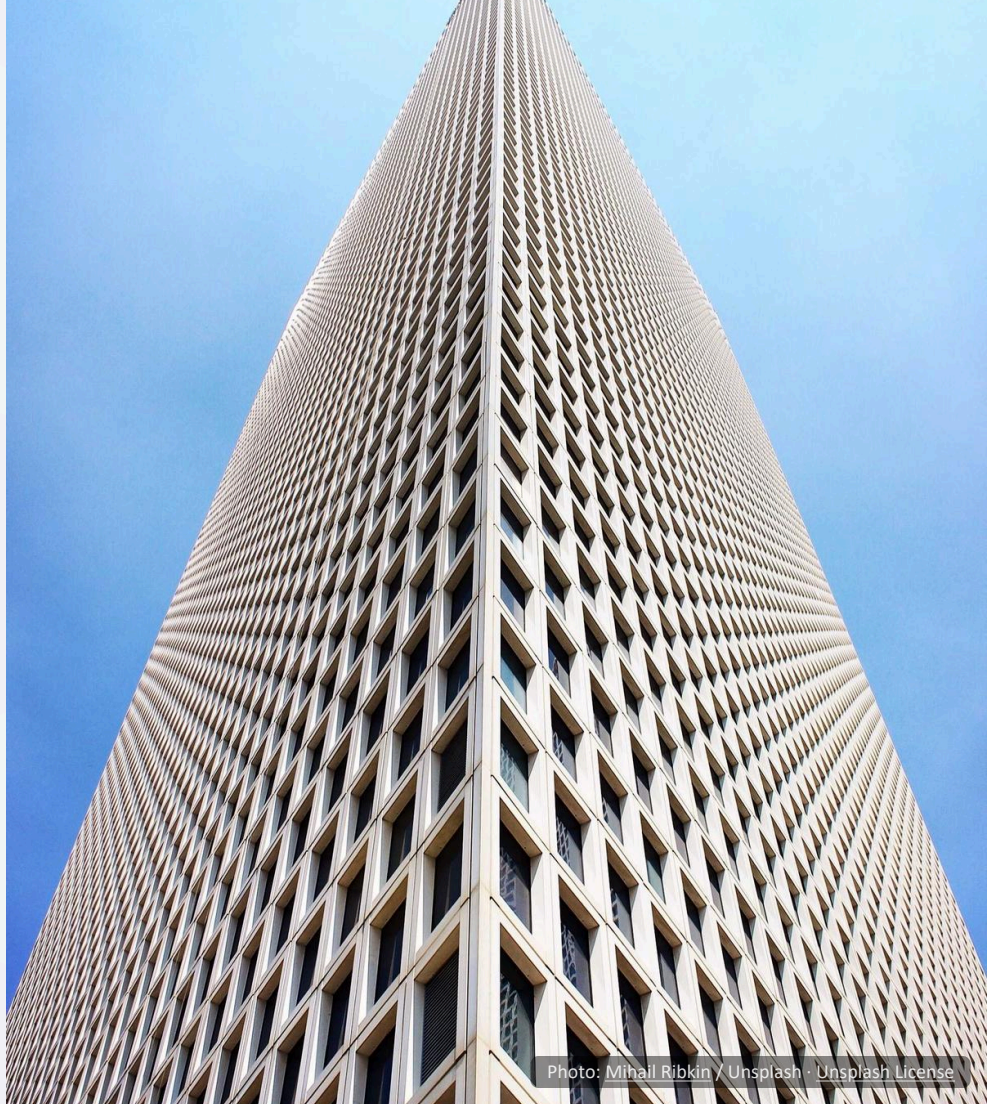
Laws and Principles

Module 2 · 40 min

NimblePros
TRAINING & MENTORING

What is software architecture?

The decisions that are hard to change later.



First Law

Everything in software architecture is a trade-off.

Source: Mark Richards & Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020)

Second Law

Why is more important than *how*.

Source: Mark Richards & Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020)

NimblePros
TRAINING & MENTORING

Encapsulation

Hide implementation details behind stable contracts.



NimblePros
TRAINING & MENTORING

Coupling

How much one part of the system depends on another.

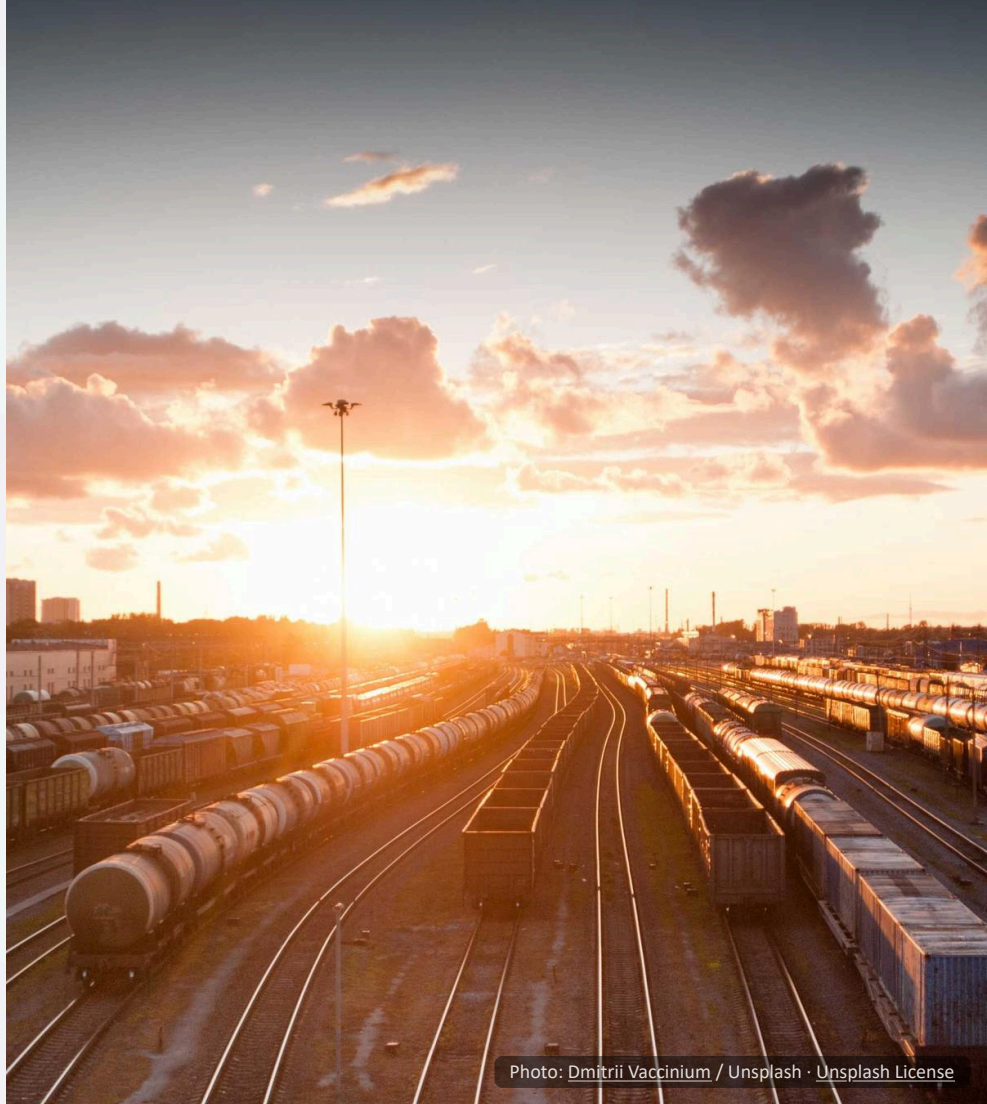
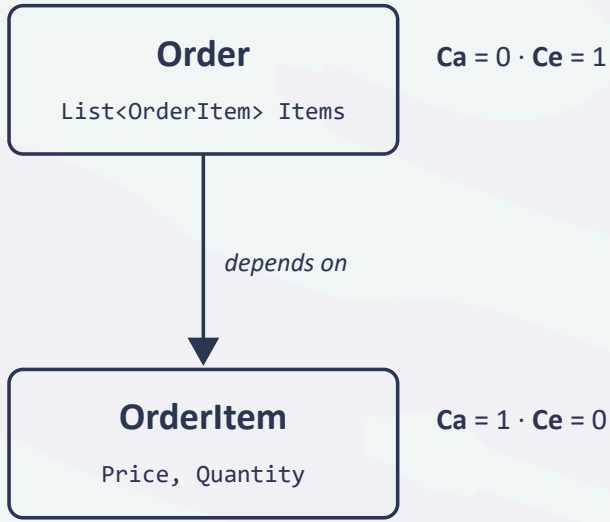


Photo: Dmitrii Vaccinium / Unsplash · Unsplash License

Afferent and efferent coupling

- **Afferent (Ca)** — incoming: who depends on me?
- **Efferent (Ce)** — outgoing: who do I depend on?



NimblePros
TRAINING & MENTORING

Connascence

Two components are connascent if a change in one requires a change in the other.



Photo: Sylwia Bartyzel / Unsplash · Unsplash License

Connascence vs. coupling

- Connascence doesn’t need a dependency arrow
- Coupling has a direction; connascence is mutual

The short version

Coupling tells you *how much* and *which way*.

Connascence tells you *what kind* and *how bad*.

Connascence dimensions

- **Static vs. dynamic** — visible in code vs. only at runtime
- **Strength** — how hard the coupling is to refactor
- **Locality** — how close the coupled elements are
- **Degree** — how many elements are affected

Prefer weaker forms, and keep strong forms close together.

Static connascence

Visible in the source code. Listed from weakest to strongest.

- **Name** — agree on a name, like a method or property
- **Type** — agree on a type
- **Meaning** (convention) — agree on what a value means, like `1` means "paid"
- **Position** — agree on an order, like positional arguments
- **Algorithm** — agree on an algorithm, like the same fee formula in two places

Dynamic connascence

Visible only at runtime. Listed from weakest to strongest.

- **Execution** — order matters, like calling `Open()` before `Send()`
- **Timing** — when things happen matters, like races and timeouts
- **Values** — several values must change together, like a total and its line items
- **Identity** — components must refer to the same instance

From position to name

```
// Connascence of position: which DateTime is the start?
// This compiles, but the arguments are swapped.
calendar.Schedule("Design review",
    new DateTime(2026, 11, 3, 15, 0, 0), new DateTime(2026, 11, 3, 14, 0, 0));

// Connascence of name: named arguments make the agreement explicit
calendar.Schedule("Design review",
    start: new DateTime(2026, 11, 3, 14, 0, 0),
    end: new DateTime(2026, 11, 3, 15, 0, 0));
```

NimblePros
TRAINING & MENTORING

Cohesion

Things that change together belong together.

The goal, at every scale:

⬆️ *high cohesion inside a component*

⬇️ *low coupling between components.*



Photo: Jassy Onyae / Unsplash · Unsplash License

Exercise: Coupling and Cohesion

Find what makes TicketTide's legacy checkout code hard to change.

Pairs · 10 + 5 min · `exercises/coupling-cohesion` · README.md

Coupling and Cohesion: steps

1. `cd exercises/coupling-cohesion`, run `dotnet run OrderManager.cs`, and read the code
2. List each problem: where, what kind, what change it makes painful
3. Check `connascence-summary.md` for kinds you missed
4. Pick the one you would fix first

Debrief

- Which problem has already caused a bug?
- Which problems exist only because of *where* the code lives?
- Which of `OrderManager`'s dependencies are volatile?



Photo: Vladimir Kudinov / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Principles at architectural scale

- Dependency inversion
- Separation of concerns

03

SECTION

"-ilities" and Trade-off Analysis

Module 3 · 65 min

NimblePros
TRAINING & MENTORING

Architecture characteristics

- The qualities a system needs beyond its features
- Also called *non-functional requirements* (NFRs) or *quality attributes*



Photo: Ksenia Kudelkina / Unsplash · Unsplash License

Three kinds of characteristics

- **Operational** — availability, scalability, performance, etc.
- **Structural** — maintainability, testability, deployability, etc.
- **Cross-cutting** — security, observability, cost, etc.

Availability

Operational · Up when users need it

- **Measure:** uptime in nines; 99.9% allows about 9 hours down a year, 99.99% about 53 minutes
- **Compared with fault tolerance:** availability is the outcome; fault tolerance is one way to get it
- **Pulls against:** cost, since each extra nine costs more, and data consistency during a network partition

Fault tolerance

Operational · Keeps working when parts of it fail

- **Measure:** what still works when a given dependency is down, and how long recovery takes
- **Compared with availability:** a means to it; you get it with redundancy, retries, fallbacks, and offline modes
- **Pulls against:** simplicity and cost; every fallback is another code path to build and test

Performance

Operational · Responds fast enough under expected load

- **Measure:** latency percentiles (p95, p99) and throughput, at a stated load
- **Compared with scalability:** performance is one point; scalability is the curve as load grows
- **Pulls against:** maintainability and data consistency, via caches, denormalization, and tuned code

Scalability

Operational · Handles more users or data by adding resources

- **Measure:** the load at which you miss your performance target, and what adding a node buys you
- **Compared with elasticity:** scalability is growth over months; elasticity is bursts over minutes
- **Pulls against:** simplicity, data consistency, and performance; scaling out means more nodes, partitions, and coordination

Elasticity

Operational · Handles sudden bursts and scales back down

- **Measure:** how fast you absorb a spike (10× load in 5 minutes) and how fast you scale back down
- **Compared with scalability:** scalability plus speed, and the scale-down is where the savings are
- **Pulls against:** simplicity and cost; autoscaling, warm capacity, queues, or a waiting room

Maintainability

Structural · Easy to change safely

- **Measure:** lead time for a typical change, and how many modules it touches
- **Compared with extensibility:** maintainability is changing what's there; extensibility is adding alongside it
- **Pulls against:** performance shortcuts and deadline pressure; you buy it with cohesion and low coupling

Testability

Structural · Easy to verify that it works

- **Measure:** how fast the suite runs, and how much logic you can test without real infrastructure
- **Compared with observability:** testability checks before release; observability tells you after
- **Pulls against:** simplicity; seams and test doubles add indirection, and distribution makes end-to-end tests hard

Deployability

Structural · Easy and low-risk to release

- **Measure:** deployment frequency, change failure rate, and time to roll back
- **Compared with maintainability:** code can be easy to change and still scary to release
- **Pulls against:** simplicity and cost; independent deployables mean more pipelines and versioned contracts

Extensibility

Structural · Easy to add new variants or features

- **Measure:** how many existing files change when you add a variant, like a new payment provider
- **Compared with customizability:** extensibility is for developers; customizability is for users, without a release
- **Pulls against:** simplicity; every extension point is an abstraction to maintain, even if nothing uses it

Customizability

Structural · Users or tenants can tailor behavior or look

- **Measure:** share of tenant requests met by configuration instead of code, and time to onboard a tenant
- **Compared with extensibility:** the user makes the change, not the developer
- **Pulls against:** testability and performance; every setting multiplies the combinations to test

Simplicity

Structural · Few moving parts for the team to understand and run

- **Measure:** number of deployables and technologies, and time for a new developer's first change
- **Compared with maintainability:** simplicity is about the whole system; maintainability is about one change
- **Pulls against:** nearly every other characteristic; it's the currency most of them are paid in

Security

Cross-cutting · Protects data and functions from misuse

- **Measure:** time to patch critical vulnerabilities, pen test findings, and compliance controls met
- **Compared with availability:** a denial-of-service attack is a security failure users see as downtime
- **Pulls against:** performance and simplicity; every check, hop, and encryption step adds latency and parts

Observability

Cross-cutting · You can tell what the system is doing and why

- **Measure:** time to detect and diagnose an incident, and share of requests you can trace end to end
- **Compared with monitoring:** monitoring says something is wrong; observability lets you ask why
- **Pulls against:** cost and performance; logs, metrics, and traces cost storage and overhead

Data consistency

Cross-cutting · Every reader sees correct, agreed-upon data

- **Measure:** anomalies like double-sold seats or report mismatches, and how stale a read can be
- **Compared with availability:** during a network partition, you choose one (CAP)
- **Pulls against:** availability, scalability, and performance; strong consistency needs coordination

Cost

Cross-cutting · Affordable to build and operate

- **Measure:** hosting cost per unit of business, like per ticket sold, plus team-months to build
- **Compared with simplicity:** related but different; a managed service can cost more money and less effort
- **Pulls against:** nearly every operational characteristic; nines, elasticity, and redundancy are all bought

How they pull against each other

Push for…	…and you usually pay with
Availability, fault tolerance	Cost, data consistency
Scalability, elasticity	Simplicity, cost, data consistency, performance
Performance	Maintainability, data consistency, scalability
Data consistency	Availability, scalability, performance
Deployability, extensibility, customizability	Simplicity, testability
Security, observability	Performance, cost

Simplicity and cost show up on almost every row.

Easy to confuse

- **Scalability** handles steady growth; **elasticity** handles sudden bursts, then scales back down
- **Performance** is speed at expected load; **scalability** is keeping it as load grows
- **Availability** is being up when needed; **fault tolerance** is staying up when parts fail

Pick the top three

Supporting every characteristic produces a system that supports none of them well.

NimblePros
TRAINING & MENTORING

Make it measurable

"Fast" isn’t a target. "p99 latency under 200 ms" is.



Photo: Sonja Langford / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

No best practices

Only trade-offs in context. *It depends.*



Aim for the "least worst" architecture

Not the best one. That doesn’t exist.

Weighted characteristics

Characteristic	Weight	Option A	Option B
Scalability	3	★★★	★★
Deployability	2	★★	★★★★
Simplicity	1	★★★★	★★
Weighted total		**17**	**16**

Score each option and make the weights visible.

TicketTide: what the stakeholders said

- **CEO:** "When a big act goes on sale, we cannot crash."
- **Partnerships:** "Venues leave us when their reports are wrong."
- **Operations:** "Scanning can’t stop working just because the network drops."
- **CFO:** "I’d rather not triple our hosting bill."
- **Lead developer:** "I’d like to be able to ship small changes safely."

Exercise: Architecture Kata

Pick and justify the top three characteristics for TicketTide.

Teams of 3–5 · 15 + 5 min · exercises/architecture-kata/README.md

Architecture Kata: steps

1. **Alone:** read the brief and highlight quality requirements
2. **Team:** list candidate characteristics with evidence from the brief
3. **Team:** pick your top three (no more!)
4. **Team:** give each one a measurable target
5. **Team:** name one characteristic you are giving up, and why

Debrief

- Which characteristic did every team pick?
- What implicit characteristics did nobody ask for?
- Do checkout and door scanning need the same characteristics?

Exercise: Compare Two Candidates

Choose between a modular monolith and event-driven microservices, and write down why.

Same teams · 10 + 5 min · exercises/compare-candidates/README.md

Compare Two Candidates: steps

1. Weight your top three characteristics 3, 2, 1 (add cost and simplicity at 1)
2. Score both candidates 1–5 on each, with a one-sentence justification
3. Multiply and total
4. Sanity-check: does the winner feel right?
5. Write the decision: "We chose ___ because ___. We accept ___."

Debrief

- Did your total pick the "wrong" winner? What did you change?
- Which single score would flip your decision?
- What did you accept as a consequence?



NimblePros · Training & Mentoring

04

SECTION

Monolithic Architecture Styles

Module 4 · 65 min

NimblePros
TRAINING & MENTORING

Layered (N-tier)

The baseline most of us started with.

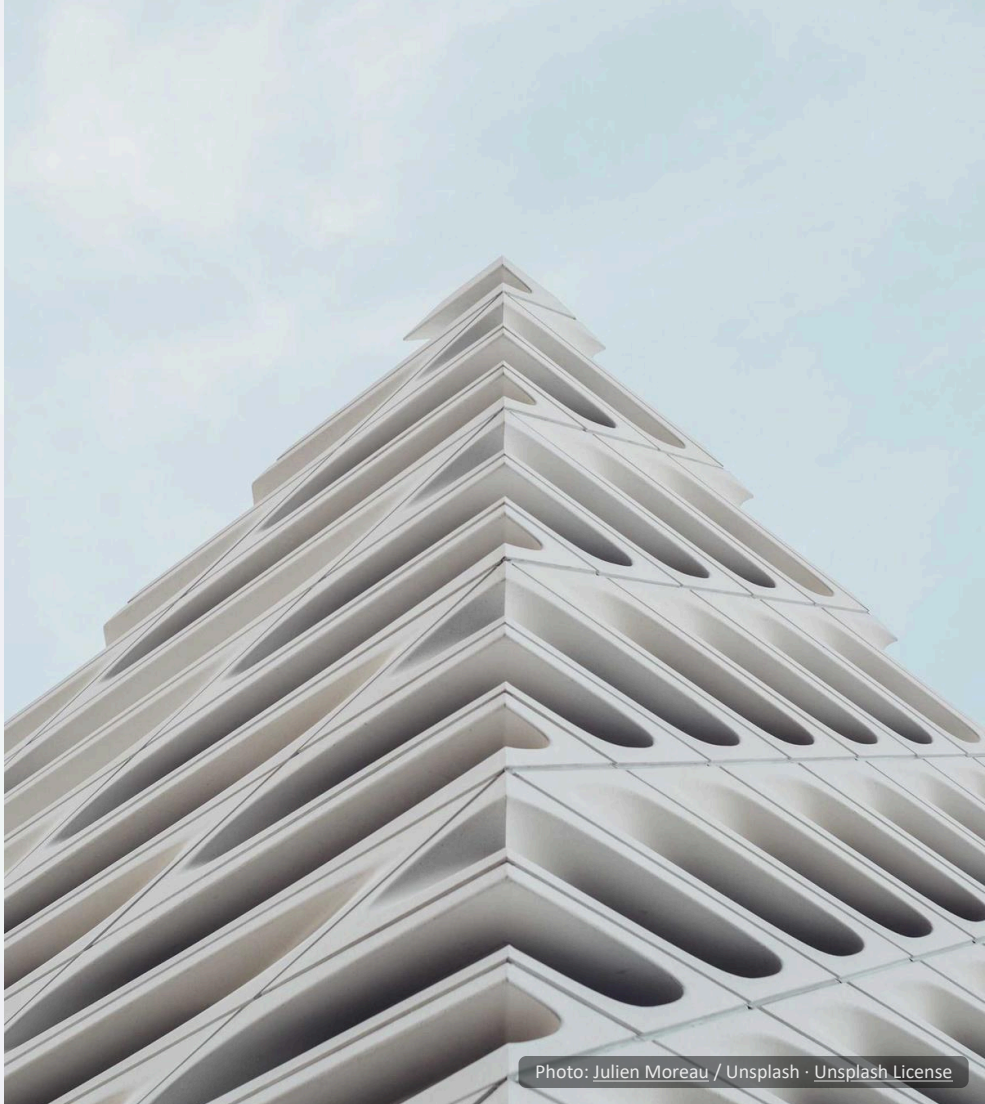
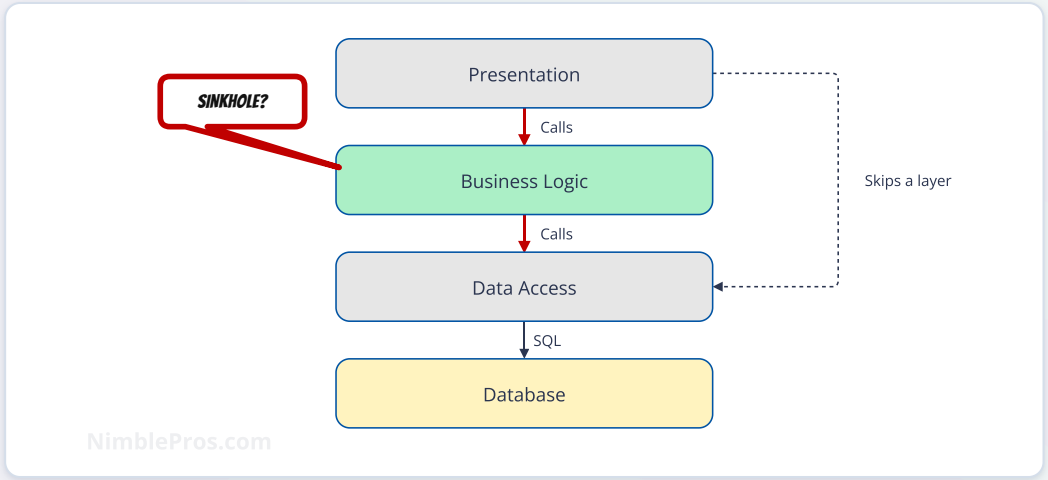


Photo: Julien Moreau / Unsplash · Unsplash License

Layered architecture



Partitioning

- **Technical partitioning** — organize by layer: UI, business, data
- **Domain partitioning** — organize by business capability: orders, billing, shipping

NimblePros
TRAINING & MENTORING

Clean Architecture

Dependencies point inward, toward the domain.



Photo: Drew Patrick Miller / Unsplash · Unsplash License

The dependency rule

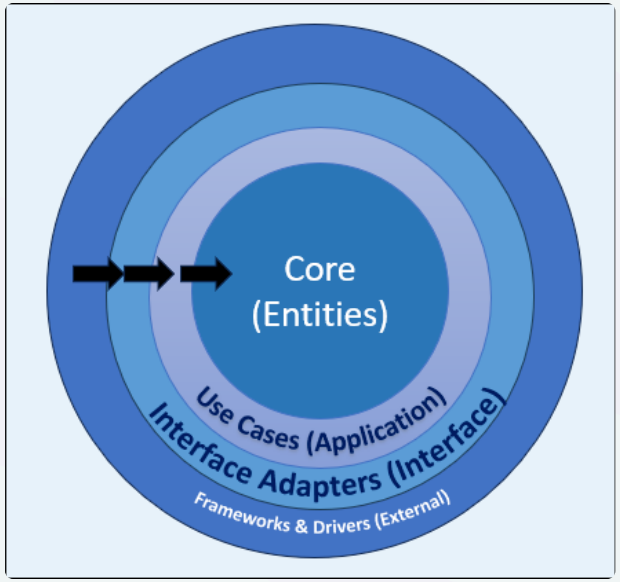


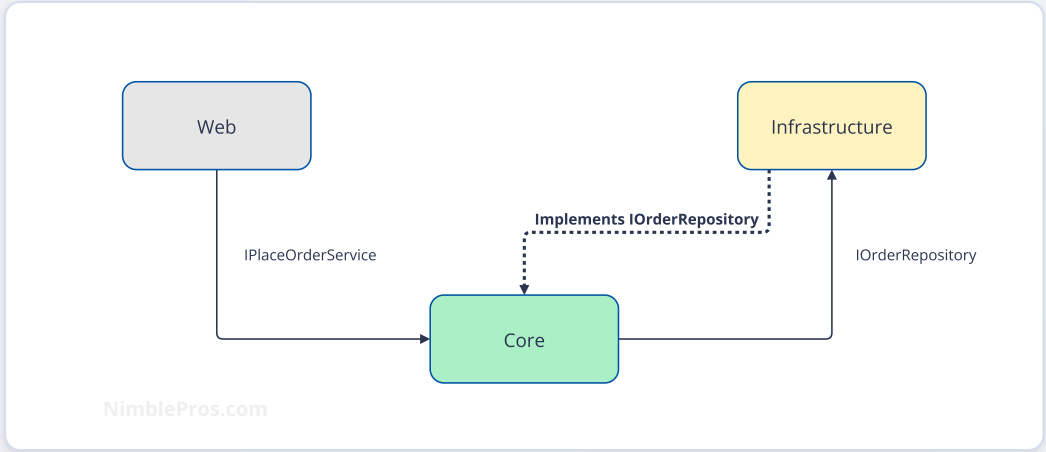
Diagram: Clean Architecture · DevIQ.com

- Source code dependencies point **inward**
- Inner circles know nothing about outer ones
- The domain is in the middle; the database and UI are on the outside
- Same idea as Hexagonal (Ports and Adapters) and Onion Architecture

Clean Architecture projects

- **Core** — domain model and interfaces
- **Infrastructure** — implements Core's interfaces
- **Web** — endpoints and composition root

Who references whom



WALKTHROUGH · 10 MIN

The Clean Architecture sample

- **Map:** the README's "Where things live" table
- **Follow** `POST /orders`: Web → Core → Infrastructure
- **Refund** the same order twice: the second call is a `400`

samples/clean-architecture

NimblePros
TRAINING & MENTORING

Vertical Slice Architecture

Organize by feature instead of by layer.

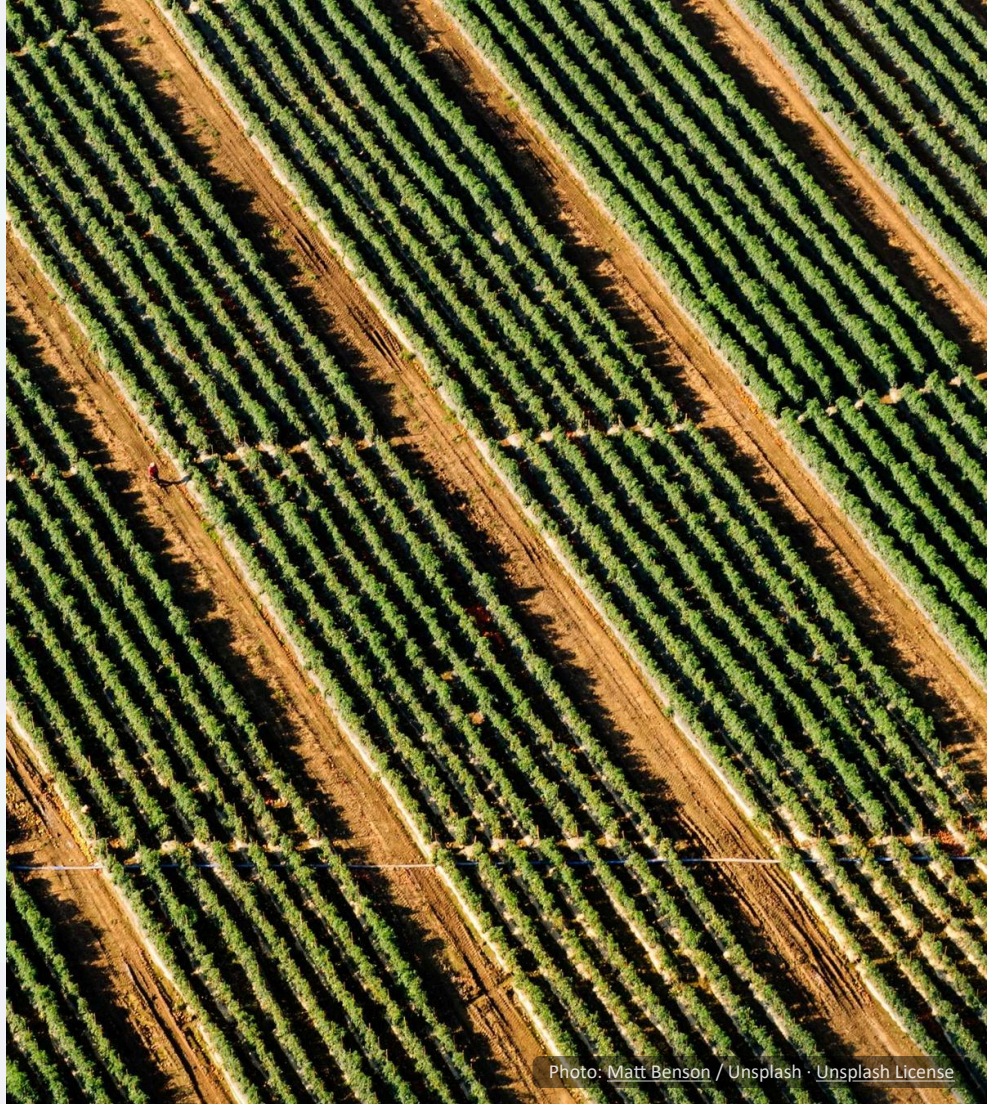
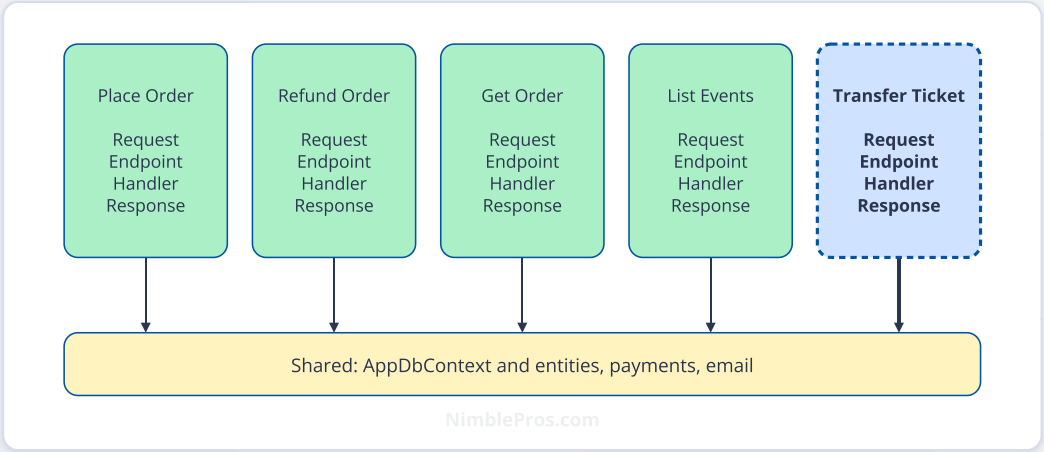


Photo: Matt Benson / Unsplash · Unsplash License

One slice per feature



WALKTHROUGH · 10 MIN

The Vertical Slice sample

- → **Open** `Features/PlaceOrder/PlaceOrder.cs`: the whole feature in one file
- → **Compare** its "Where things live" table with Clean Architecture's
- → **Same** API and the same end-to-end tests

samples/vertical-slice

NimblePros
TRAINING & MENTORING

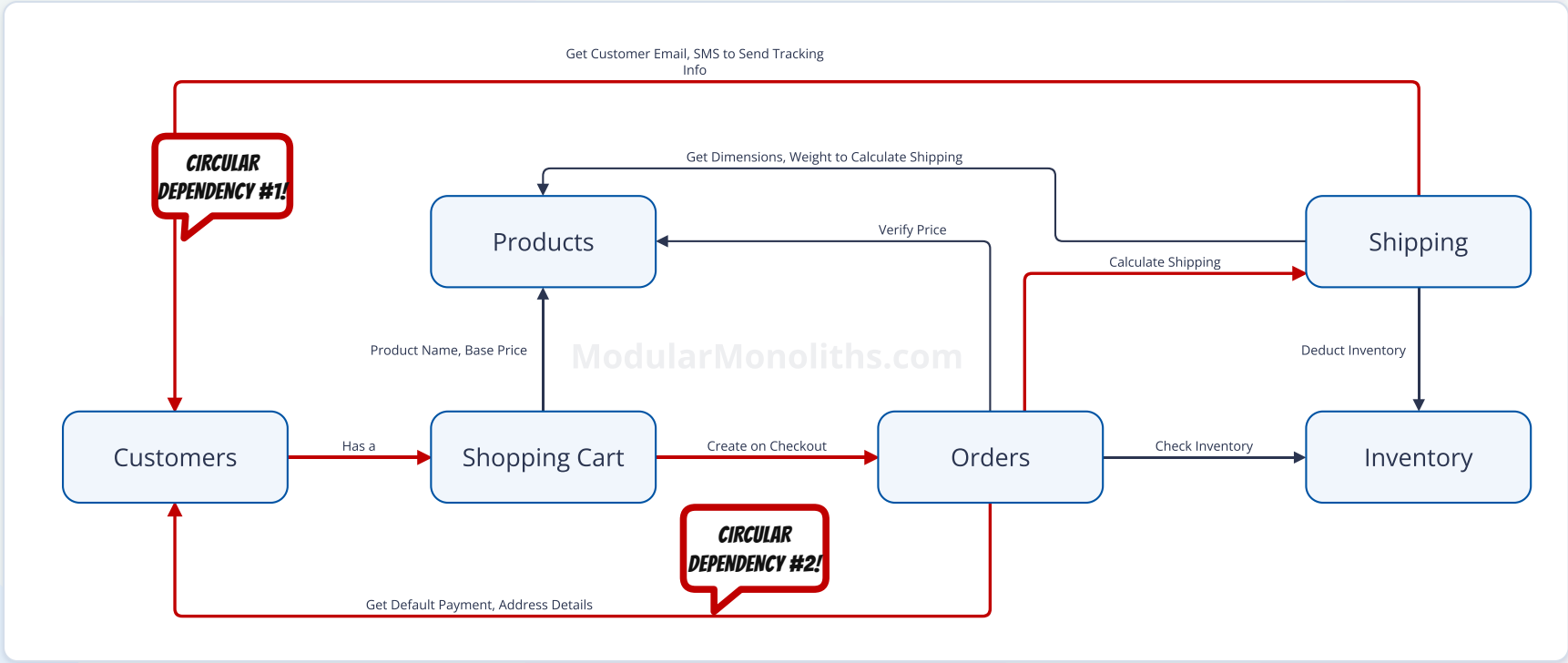
Modular monoliths

Bounded contexts as modules inside one deployable.

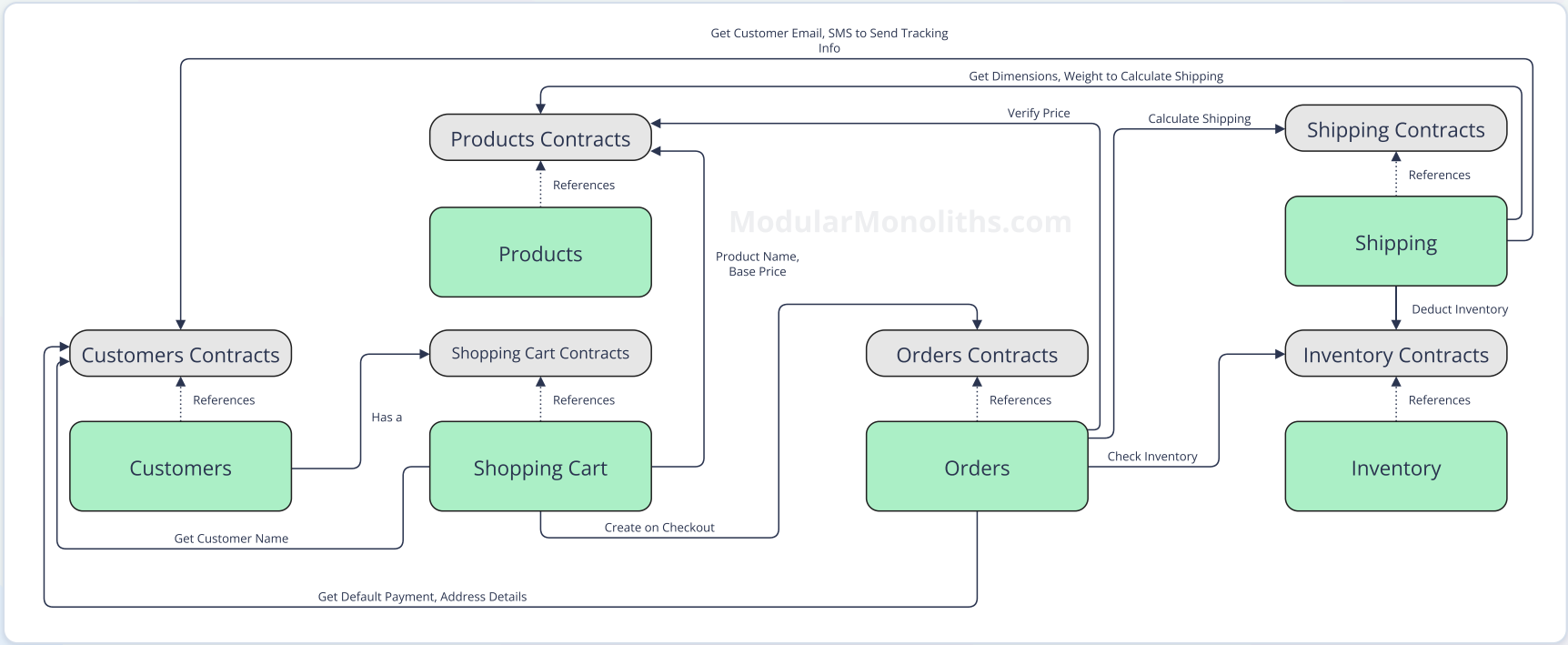


Photo: Vladimir Kudinov / Unsplash · Unsplash License

Modules without boundaries



Modules and contracts



Module boundaries

- Public contracts between modules
- In-process messaging instead of direct calls

Comparing the styles

	Clean	Vertical Slice	Modular Monolith
Partitioning	Technical	Feature	Domain
Change locality	★★	★★★★	★★★
Domain modeling	★★★★	★★	★★★
Path to extraction	★★	★★	★★★★

Exercise: Add a Feature

Add ticket transfer to one sample, then compare with a team that used the other style.

Pairs · 20 + 5 min · `samples/clean-architecture` or `samples/vertical-slice` · add-a-feature.md

The feature: ticket transfer

- `POST /orders/{orderId}/tickets/{ticketId}/transfer` with `{ "toEmail": "..." }`
- Only **paid** tickets, and **at most twice** per ticket
- The new holder gets an email; the old ticket code stops working
- `200` with the order, `404` if not found, `400` if a rule is broken

Add a Feature: steps

1. Run `dotnet BuildTestFormat.cs <your sample>` so you know the tests pass
2. Implement transfer in your sample
3. Count files touched, files added, and projects touched
4. Note where each business rule ended up
5. Pair up with a team that used the other sample and compare

Debrief

- Where did the "at most twice" rule live in each sample?
- Which style made you touch more layers? Was that useful?
- Which would make extracting transfer into a service easier?



NimblePros · Training & Mentoring

05

SECTION

Distributed Systems Fundamentals

Module 5 · 35 min

NimblePros
TRAINING & MENTORING

Why distribute?

Independent scaling, deployment, and fault isolation, at a price.



Photo: Matthew Wiebe / Unsplash · Unsplash License

Fallacies of distributed computing

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn’t change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous

NimblePros
TRAINING & MENTORING

Synchronous communication

HTTP and gRPC: simple to reason about, but the caller waits.



Photo: RayBay / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Asynchronous communication

Queues and events: the caller moves on, and the work happens later.



Photo: Alex Perz / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Temporal coupling

If B must be up for A to work, a failure in B becomes a failure in A.



Photo: petradr / Unsplash · Unsplash License

CAP

During a network partition, choose consistency or availability.

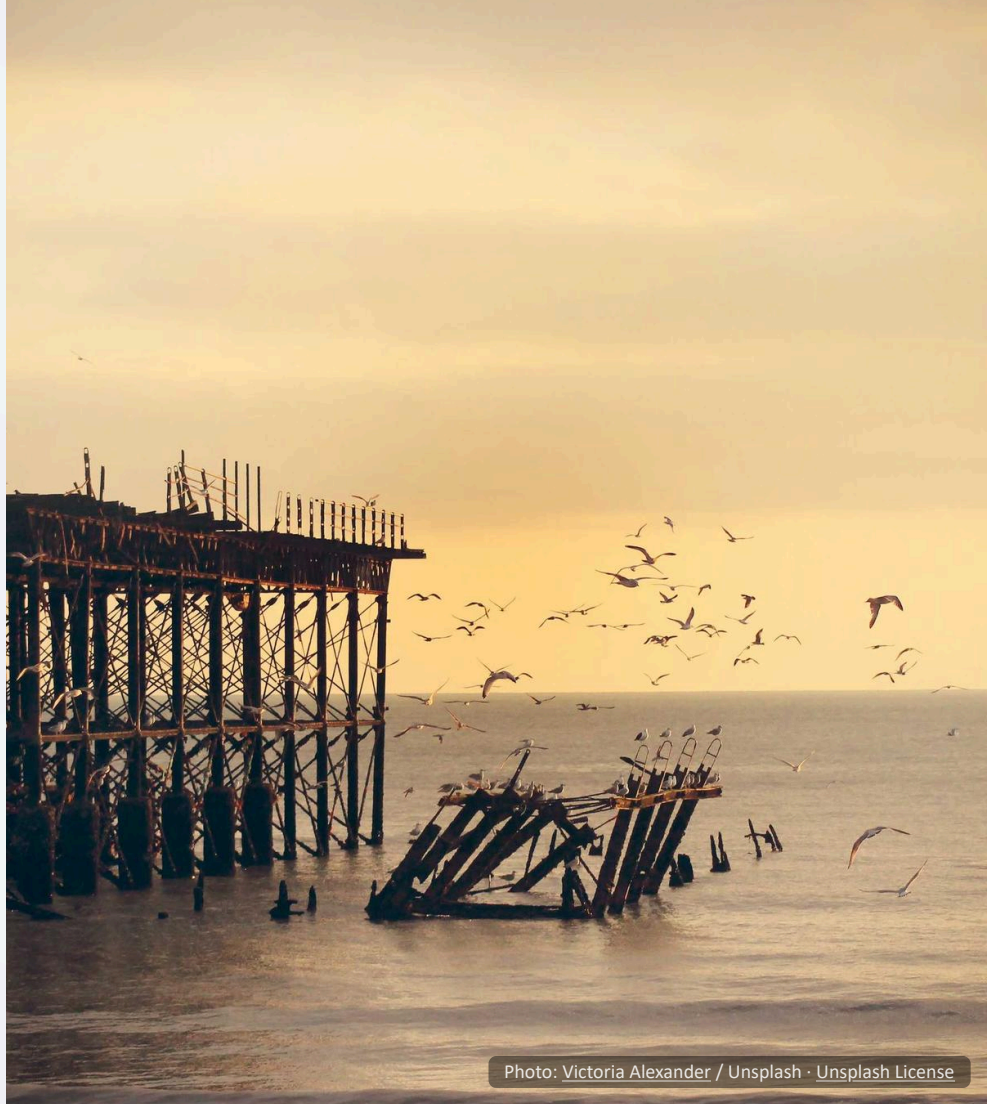
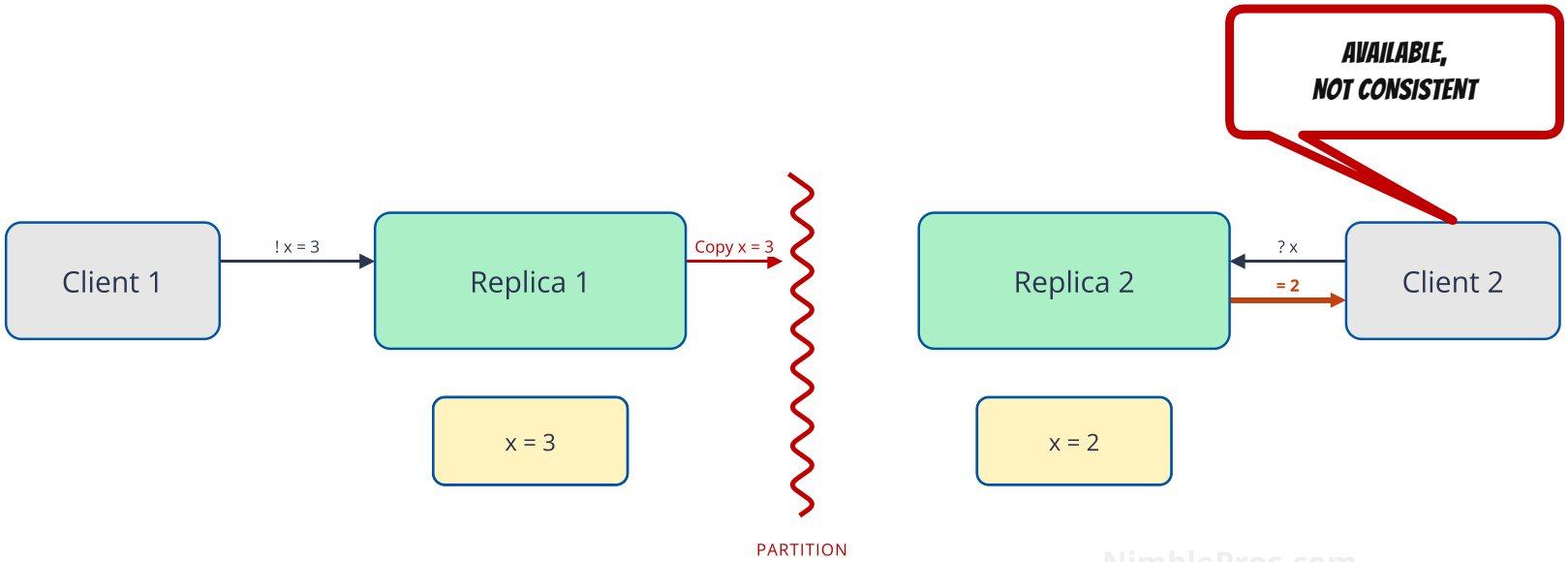


Photo: Victoria Alexander / Unsplash · Unsplash License

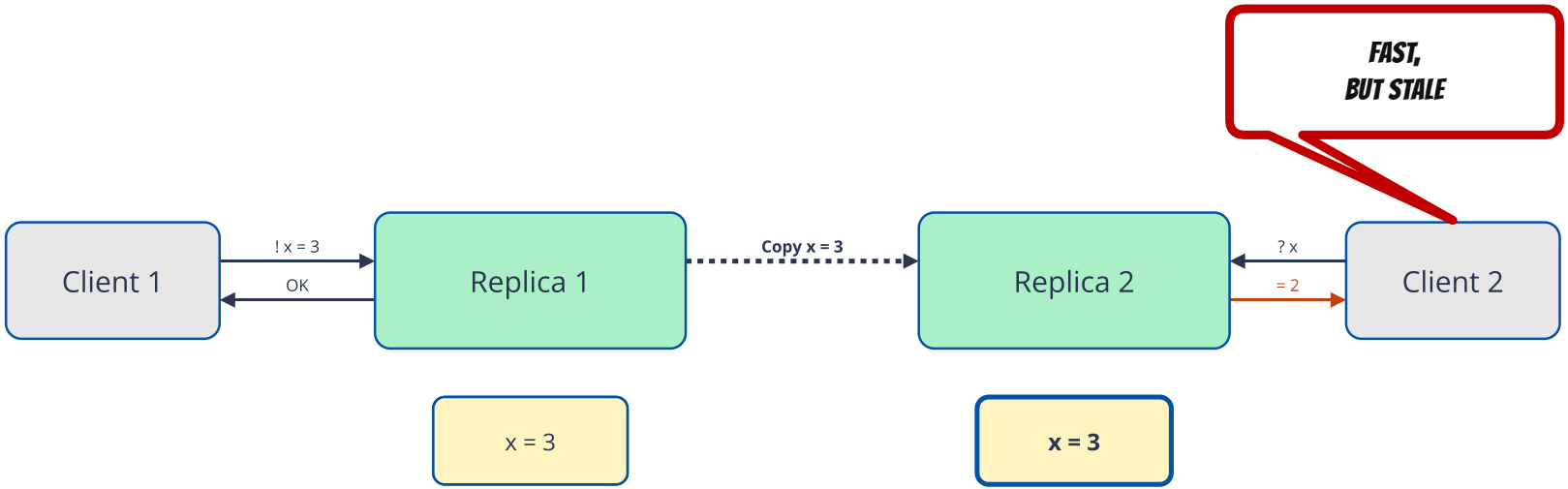
CAP: a partition forces a choice



PACELC

- If **P**artitioned, choose **A**vailability or **C**onsistency
- **E**lse, choose **L**atency or **C**onsistency

PACELC: no partition, still a trade-off



Consistency guarantees

- **Eventual** — replicas converge over time
- **Quorum** — `R + W > N` guarantees overlap
- **Strong** — every read sees the latest write, at a cost

A dependency is down. Now what?

Is it a read, or a write?

NimblePros
TRAINING & MENTORING

On read: serve from a cache

Users stay up, but may see stale data.



Photo: Patrick Tomasso / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

On write: queue the command

Accept the request and process it later.



Photo: Kim Daniel / Unsplash · Unsplash License

While the user waits: retry, briefly

- **Timeout**: never wait forever; pick a budget the user can live with
- **Retry with backoff and jitter**: a few tries, spaced out and randomized
- **Circuit breaker**: after repeated failures, stop calling and fall back at once
- Retry only what's safe to repeat: reads, or writes with an idempotency key

Discussion: A Dependency Is Down

Decide what TicketTide does when something it depends on fails.

Small groups · 8 + 5 min · scenarios 1 and 2 · dependency-failures.md

A Dependency Is Down: steps

1. Take one scenario: the seat map service or the payment provider
2. Is it a read problem, a write problem, or both?
3. Retry, cache, queue, or fail fast? What do you tell the user?
4. Which characteristic are you trading away?

Debrief

- Which scenarios could use a cache, and which never could?
- What makes a queued retry safe?
- Did the CAP choice differ within one system?

NimblePros
TRAINING & MENTORING

Afternoon

Services, Decisions, and Governance



NimblePros · Training & Mentoring

06

SECTION

SOA, DDD, and Microservices

Module 6 · 15 min

NimblePros
TRAINING & MENTORING

Service-Oriented Architecture

Entity services, shared databases, and the enterprise service bus.

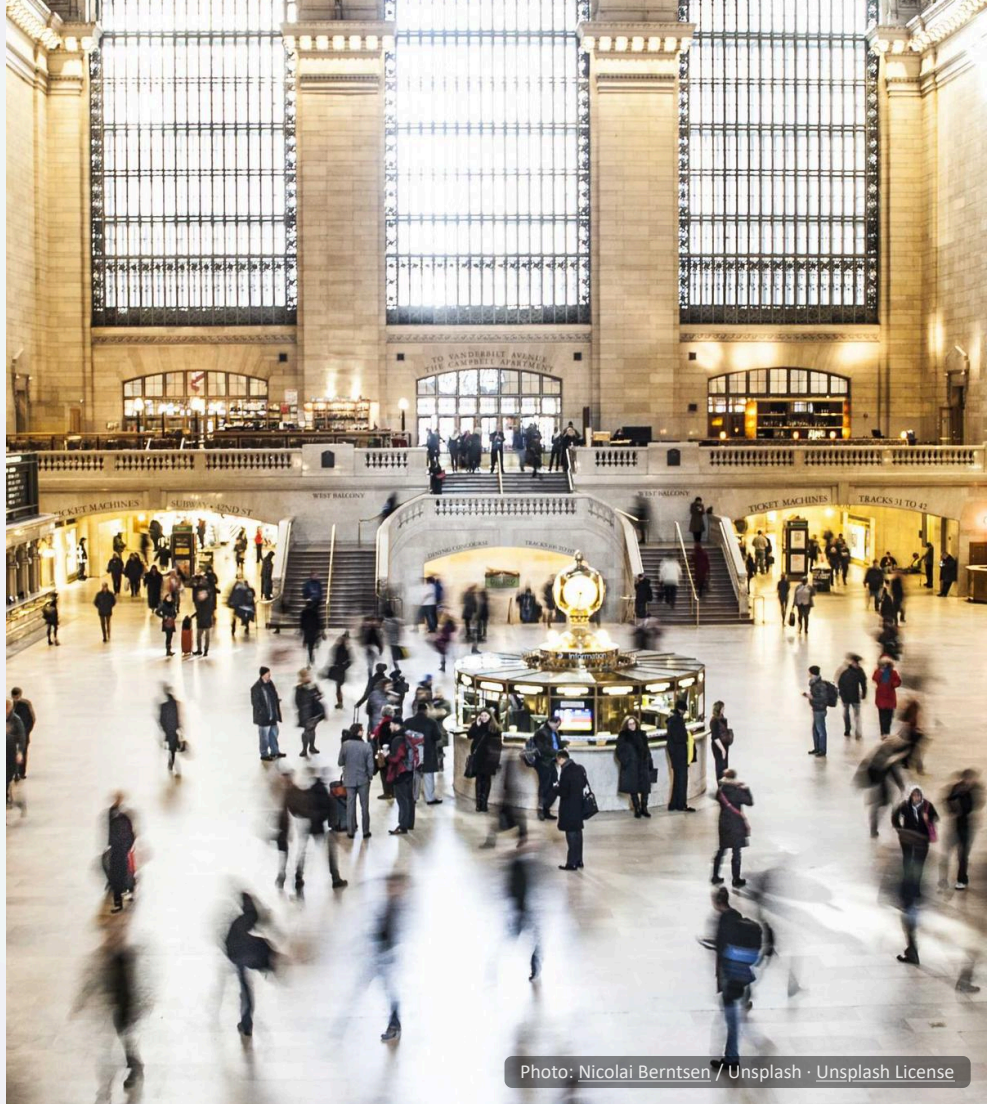


Photo: Nicolai Berntsen / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Ubiquitous language

Use the same words in conversations, code, and tests.



Photo: mr.lee / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Bounded contexts

A boundary inside which a model and its language are consistent.

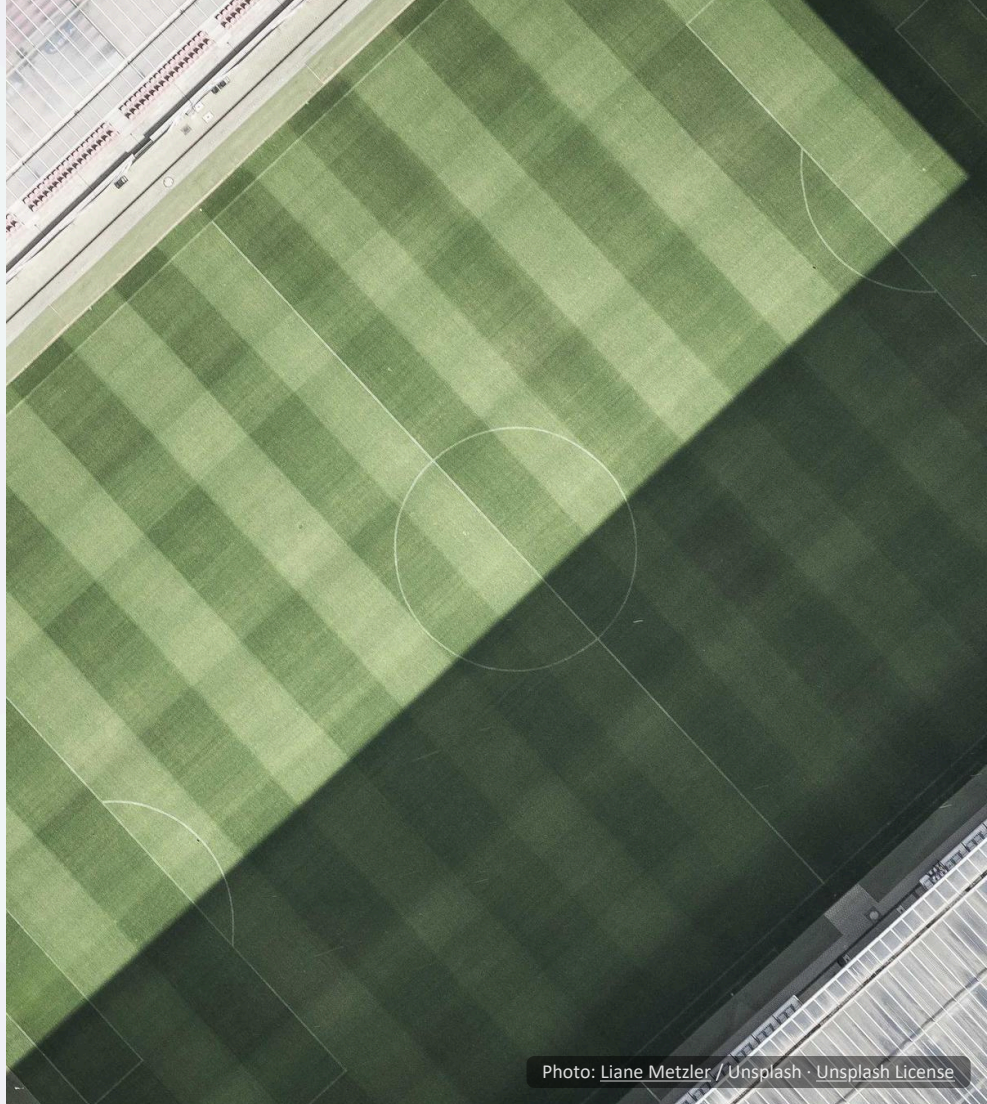


Photo: Liane Metzler / Unsplash · Unsplash License

Anti-corruption layers

Protect your model from someone else’s.



Photo: Art Wave / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Microservices

One service per bounded context, each with its own database.



NimblePros
TRAINING & MENTORING

Avoid reach-in reporting

Push events into a reporting store instead of querying service databases.



Photo: Joanna Kosinska / Unsplash · Unsplash License

07

SECTION

Modularity vs. Granularity

Module 7 · 30 min

NimblePros
TRAINING & MENTORING

Modularity

Breaking a system into parts.

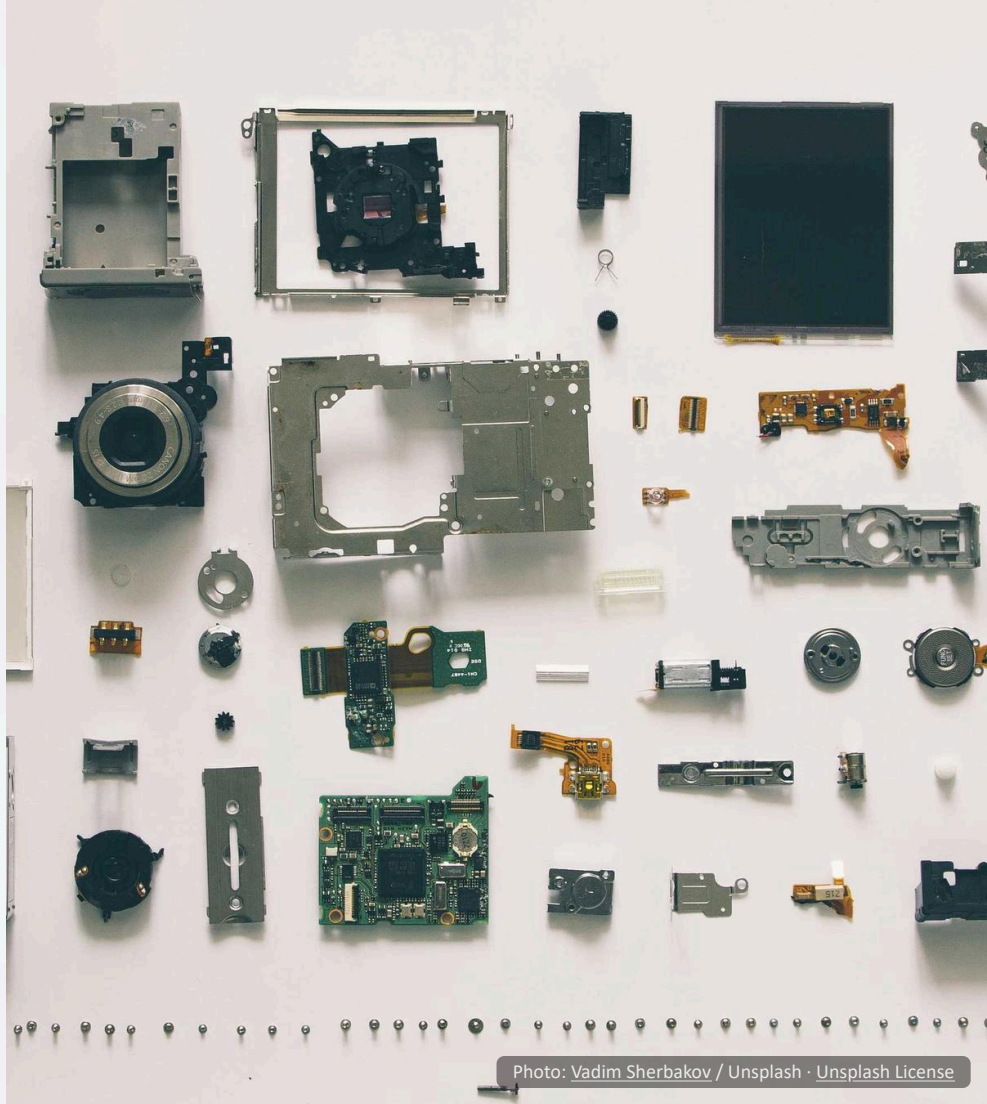


Photo: Vadim Sherbakov / Unsplash · Unsplash License

Granularity

The size of each service: how much functionality and data it owns.

Modularity draws the seams. **Granularity** decides which seams become network boundaries.

A modular monolith: many modules, one service. Very modular, and coarse-grained.

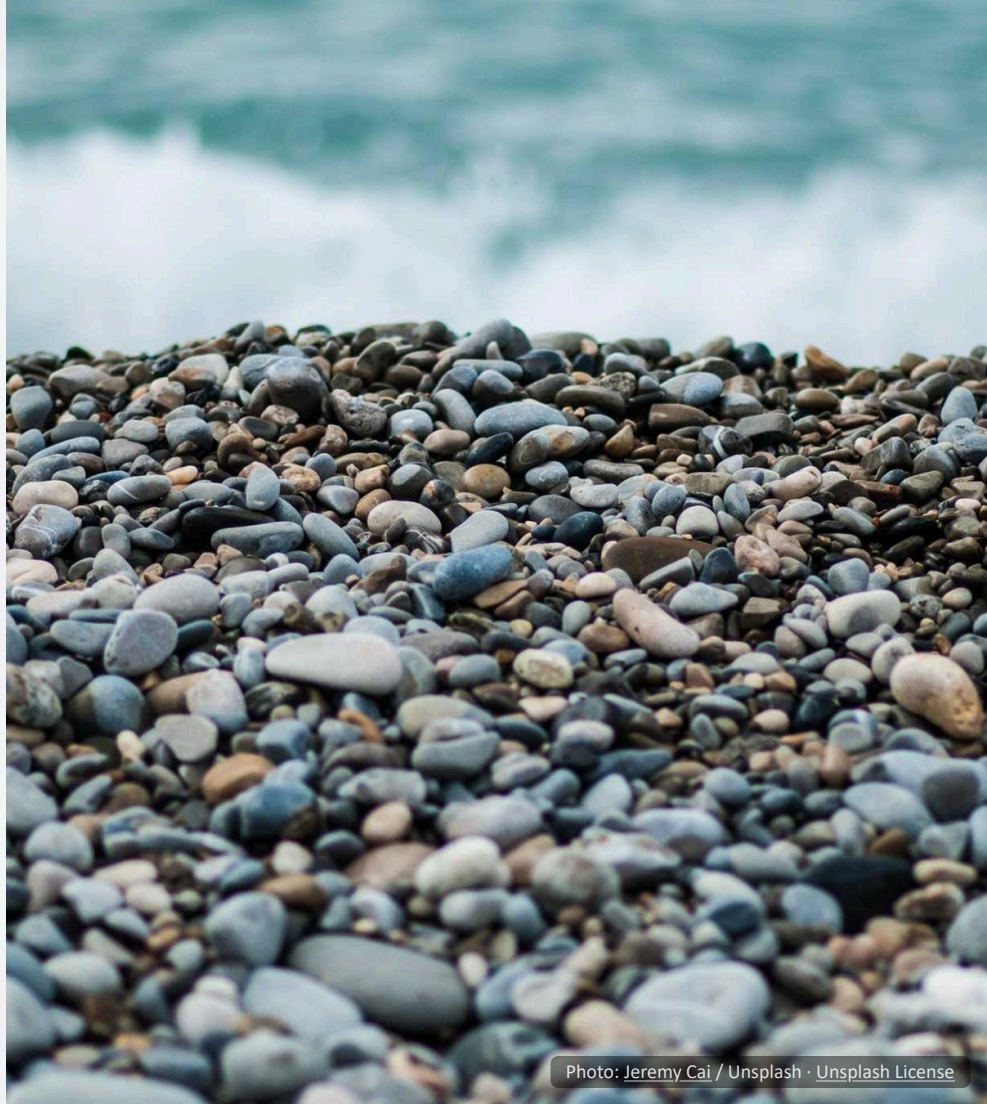


Photo: Jeremy Cai / Unsplash · Unsplash License

Coarse vs. fine-grained services

- **Fine-grained:** independent deployment and scaling, but more network calls and distributed transactions
- **Coarse-grained:** simpler transactions and workflows, but shared scaling and failure

Granularity disintegrators (not ingrators)

Forces that push toward smaller services

- Service scope and function
- Code volatility
- Scalability and throughput
- Fault tolerance
- Security
- Extensibility

Granularity integrators

Forces that push toward larger services

- Database transactions
- Workflow and choreography
- Shared code
- Data relationships

Discussion: Split or Merge?

Should TicketTide extract Inventory? Split Ordering from Payments?

Teams of 3–5 · 12 + 5 min · decisions 1 and 2 · split-or-merge.md

Split or Merge?: steps

1. List the disintegrators that apply
2. List the integrators that apply
3. Decide: split, merge, or leave it
4. Name the cost you are accepting

Debrief

- Which decision had strong forces in both directions?
- Did anyone solve the problem without changing granularity?
- Which decisions need the business, not just the team?



NimblePros · Training & Mentoring

08

SECTION

Architecture Decision Records

Module 8 · 30 min

NimblePros
TRAINING & MENTORING

What is an ADR?

A short document that captures one architectural decision.



Photo: Florian Klauer / Unsplash · Unsplash License

ADR sections

- **Context** — the forces at play
- **Decision** — what we chose
- **Status** — proposed, accepted, superseded
- **Consequences** — what happens because of it

NimblePros
TRAINING & MENTORING

Append-only

Don’t edit old decisions. Supersede them with new ones.



Photo: Dyaa Eldin Moustafa / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Preserve the why

For future team members and AI agents.



Photo: Alejandro Escamilla / Unsplash · Unsplash License

NimblePros
TRAINING & MENTORING

Keep them with the code

Store them in the repo, e.g., `docs/adr`.



Photo: Rayi Christian Wicaksono / Unsplash · Unsplash License

Common templates

- Michael Nygard’s original template
- MADR (Markdown Any Decision Records)
- Y-statements

Exercise: Write an ADR

Turn the morning’s architecture choice into an ADR.

Same teams · 12 + 3 min · `exercises/templates/adr-template.md` · write-an-adr.md

Write an ADR: steps

1. Start from your trade-off decision statement
2. Fill in the template: context, decision, consequences
3. Write one `AGENTS.md` line that protects the decision
4. Ask your agent to undo the decision: does it push back?

Debrief

- What was hardest to write?
- Did your agent catch the conflict?
- Could a new team member explain the *why* from your ADR alone?
- What would trigger superseding it?



NimblePros · Training & Mentoring

09

SECTION

Governance and Fitness Functions

Module 9 · 40 min

NimblePros
TRAINING & MENTORING

Enforce the rules

Decisions that aren’t enforced eventually stop being decisions.

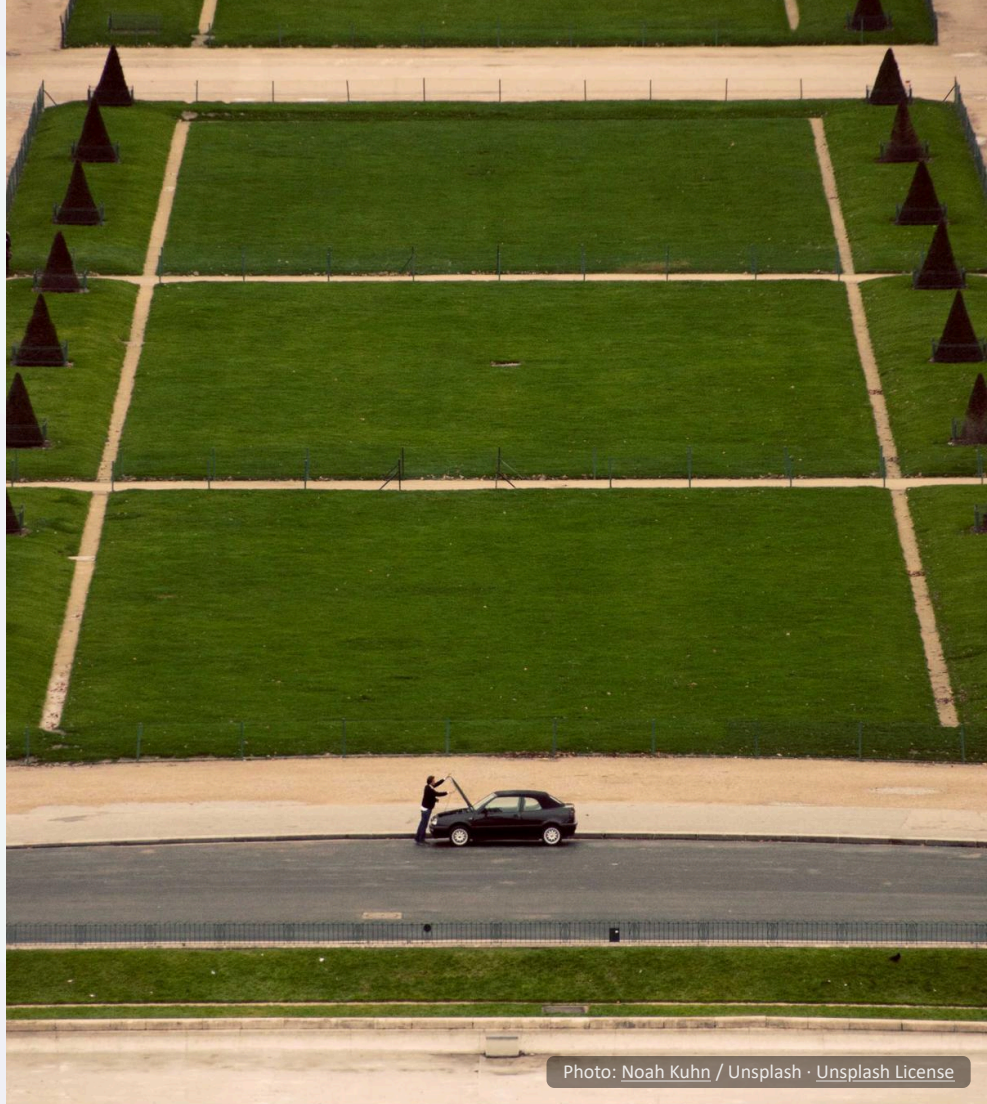


Photo: Noah Kuhn / Unsplash · Unsplash License

Enforcement tools

- Architecture tests (NetArchTest, ArchUnitNET)
- Roslyn analyzers
- Documented, time-boxed exceptions via ADRs

NimblePros
TRAINING & MENTORING

Guardrails for agents

AI agents need explicit, automated constraints.

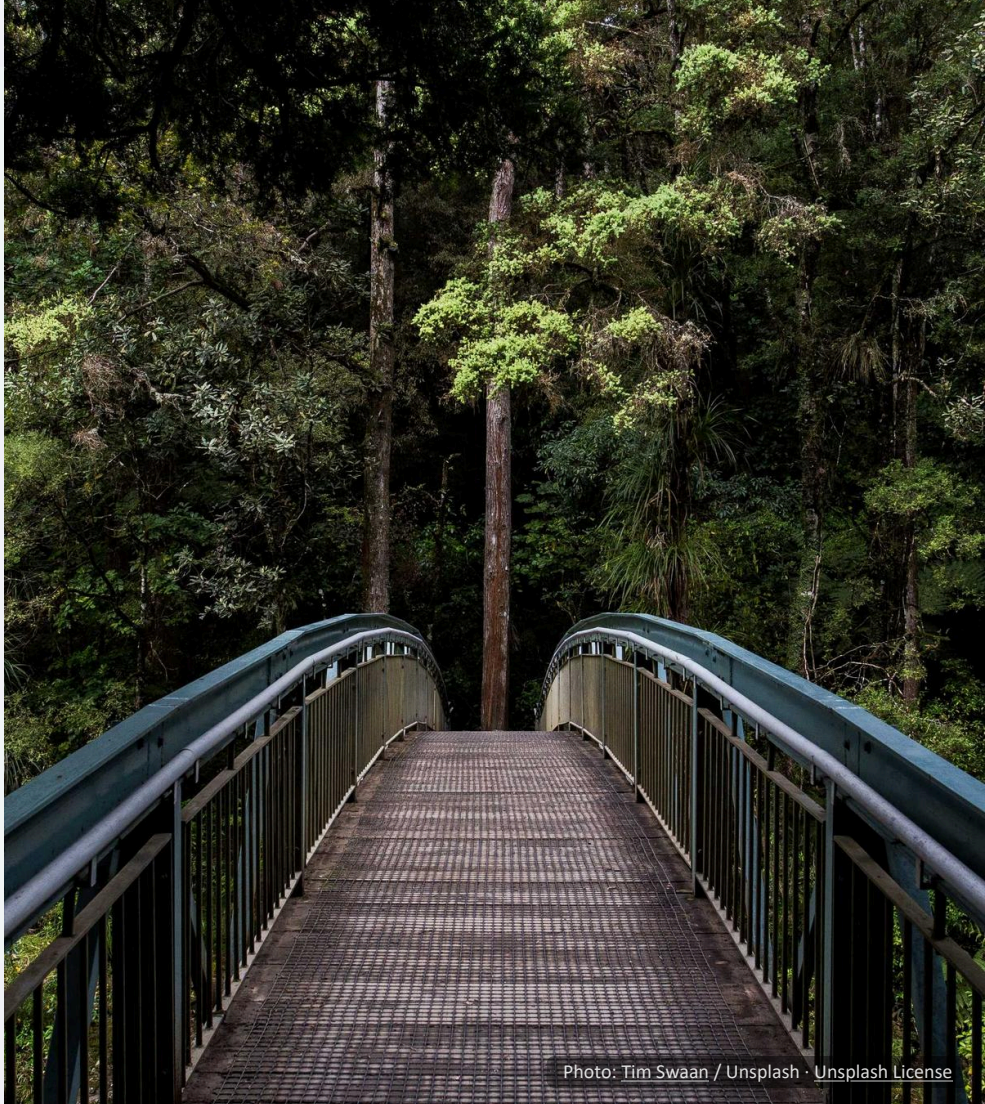


Photo: Tim Swaan / Unsplash · Unsplash License

Keeping agents in check

- `AGENTS.md` and `CLAUDE.md` for context
- Skills for repeatable workflows
- Hooks and automated checks that fail on drift

NimblePros
TRAINING & MENTORING

Fitness functions

Objective checks that an architecture characteristic still holds.



Photo: Todd Quackenbush / Unsplash · Unsplash License

Kinds of fitness functions

- **Atomic vs. holistic** — one characteristic, or several together
- **Triggered vs. continual** — on an event like a build, or all the time

An architecture test

```
[Fact]
public void Core_should_not_depend_on_Infrastructure()
{
    var result = Types.InAssembly(typeof(Order).Assembly)
        .ShouldNot()
        .HaveDependencyOn("MyApp.Infrastructure")
        .GetResult();

    Assert.True(result.IsSuccessful);
}
```

Run them everywhere

In CI for structure. In production for runtime characteristics.



Photo: davide ragusa / Unsplash · Unsplash License

Exercise: Fitness Functions

Turn architecture decisions into checks that fail the build.

Pairs · 20 + 5 min · rules 1 and 2 · `exercises/fitness-functions` · README.md

Fitness Functions: steps

1. `cd exercises/fitness-functions` and run `dotnet run fitness.cs`
2. Add two rules: Core must not depend on Infrastructure, and repositories live in Infrastructure
3. Run again: which types fail? Commit your rules
4. Ask your agent to make it pass: did it change code or rules?
5. Fix the code, not the rules

Debrief

- Which rules are atomic, holistic, triggered, or continual?
- A timing rule passes on your laptop. Would it be reliable in CI?
- Did your agent fix the code or edit a rule? Would you notice in a big PR?

In real projects: NsDepCop

Namespace rules in a config file, checked by the compiler. Not for file-based apps like `fitness.cs`.

```
<!-- RiverBooks: src/RiverBooks.OrderProcessing/config.nsdepcop -->
<NsDepCopConfig>
  <Allowed From="*" To="*" />

  <!-- Domain is the core; it should not depend on outer layers -->
  <Disallowed From="RiverBooks.OrderProcessing.Domain.*"
              To="RiverBooks.OrderProcessing.Infrastructure.*" />
  <Disallowed From="RiverBooks.OrderProcessing.UseCases.*"
              To="RiverBooks.OrderProcessing.Infrastructure.*" />

  <AnalyzerConfig>
    <IssueKind Name="IllegalDependency" Severity="Error" />
  </AnalyzerConfig>
</NsDepCopConfig>
```



NimblePros · Training & Mentoring

10

SECTION

Saga Patterns

Module 10 · 30 min

Transactions don’t distribute

- In one database, a transaction makes the work all or nothing
- Database per service: no transaction spans the services
- Distributed transactions (two-phase commit) block, and couple every service’s availability
- A **saga** is a series of local transactions, with a plan for when one fails

The three Cs

- **Communication:** synchronous (the caller waits) or asynchronous (messages, nobody waits)
- **Consistency:** atomic (all or nothing) or eventual (retry until the data catches up)
- **Coordination:** orchestrated (one service runs the workflow) or choreographed (each service knows the next step)

Two choices, three times: eight sagas.

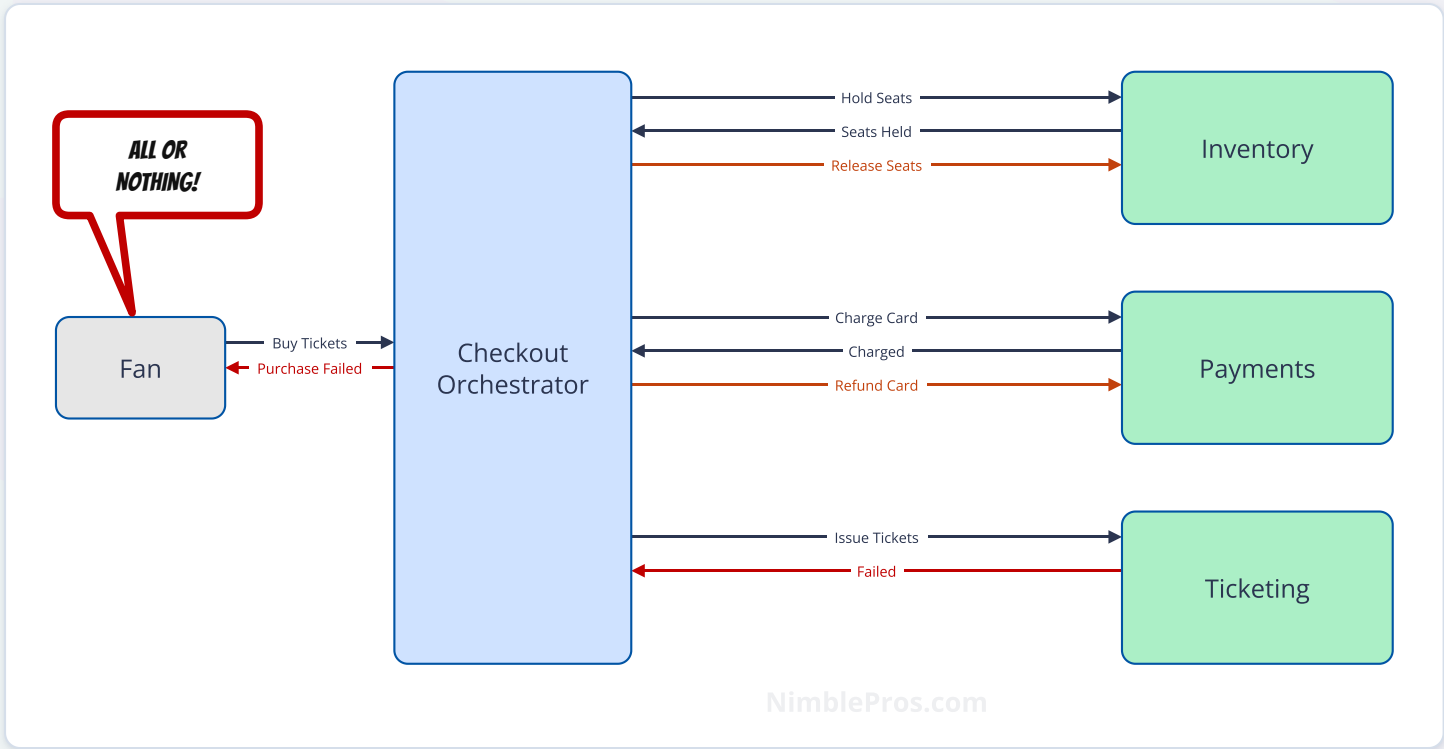
Eight sagas

Saga	Communication	Consistency	Coordination
Epic	Synchronous	Atomic	Orchestrated
Phone Tag	Synchronous	Atomic	Choreographed
Fairy Tale	Synchronous	Eventual	Orchestrated
Time Travel	Synchronous	Eventual	Choreographed
Fantasy Fiction	Asynchronous	Atomic	Orchestrated
Horror Story	Asynchronous	Atomic	Choreographed
Parallel	Asynchronous	Eventual	Orchestrated
Anthology	Asynchronous	Eventual	Choreographed

Epic Saga

- **Synchronous, atomic, orchestrated**
- An orchestrator calls each service in turn and waits for each answer
- On failure, it sends compensations, newest first, then answers the fan
- Acts like a monolith’s transaction, but every service must be up
- Sample: `samples/epic-saga`

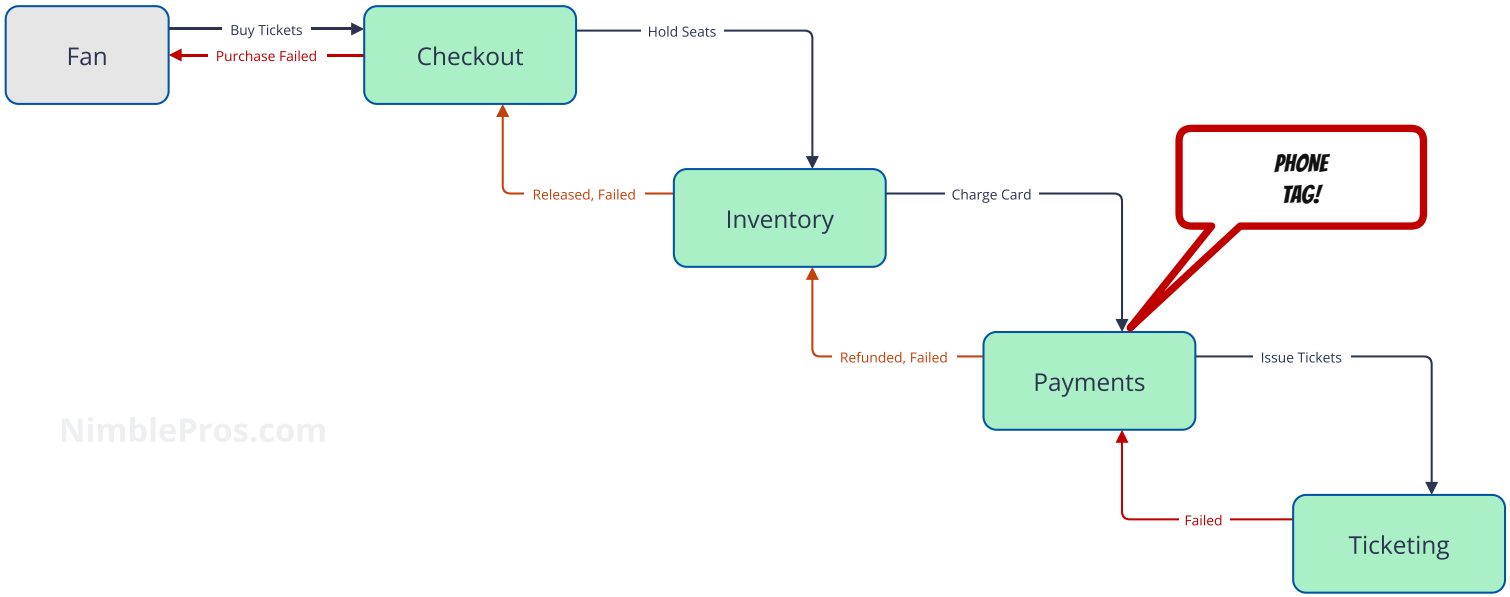
Epic Saga in action



Phone Tag Saga

- **Synchronous, atomic, choreographed**
- Each service calls the next one and waits for its answer
- A failure travels back along the chain; each service undoes its own step
- No orchestrator, but every service carries workflow and undo logic
- Sample: `samples/phone-tag-saga`

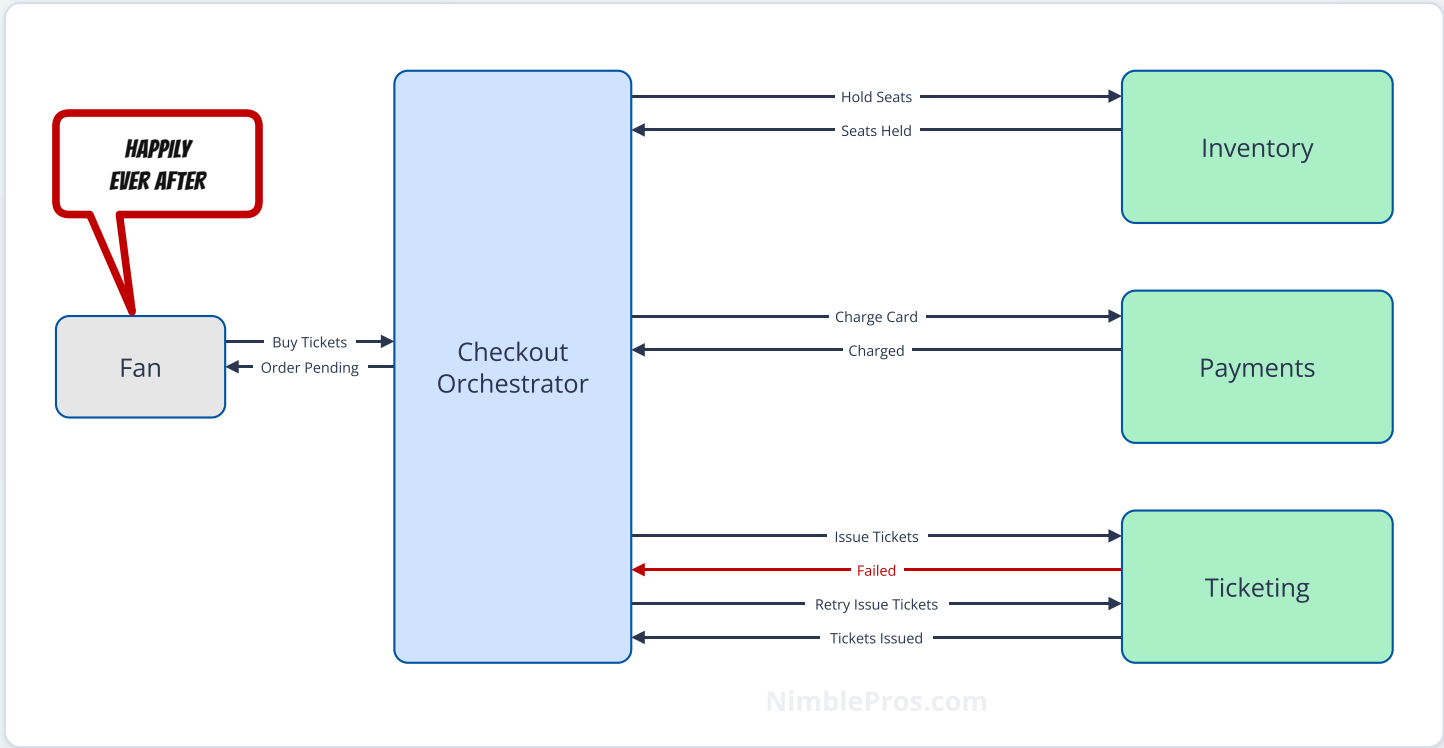
Phone Tag Saga in action



Fairy Tale Saga

- **Synchronous, eventual, orchestrated**
- An orchestrator calls each service; each commits its own work
- On failure, nothing is undone: the fan hears "pending", and the orchestrator retries
- One owner for the workflow and its errors, with much less coupling
- Sample: `samples/fairy-tale-saga`

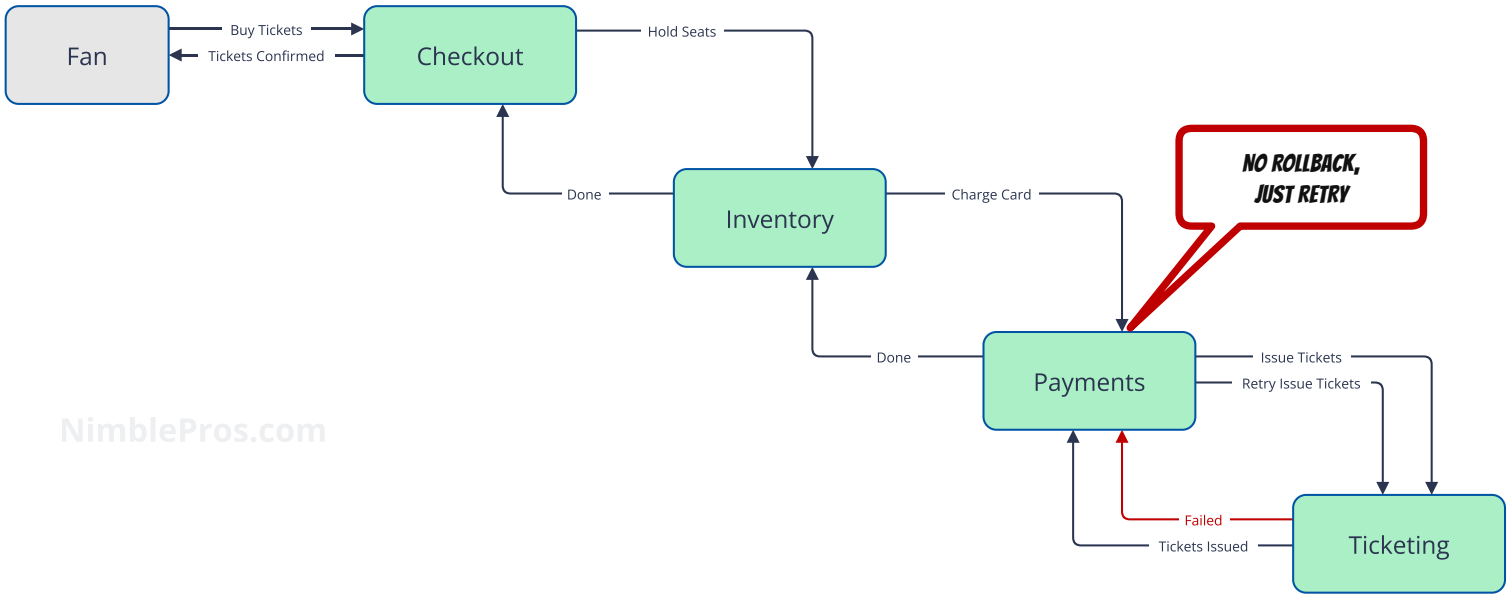
Fairy Tale Saga in action



Time Travel Saga

- **Synchronous, eventual, choreographed**
- Each service commits its work, then calls the next, like pipes and filters
- On failure, nothing is undone: the caller retries the next step
- High throughput for simple chains; hard to follow as they grow
- Sample: `samples/time-travel-saga`

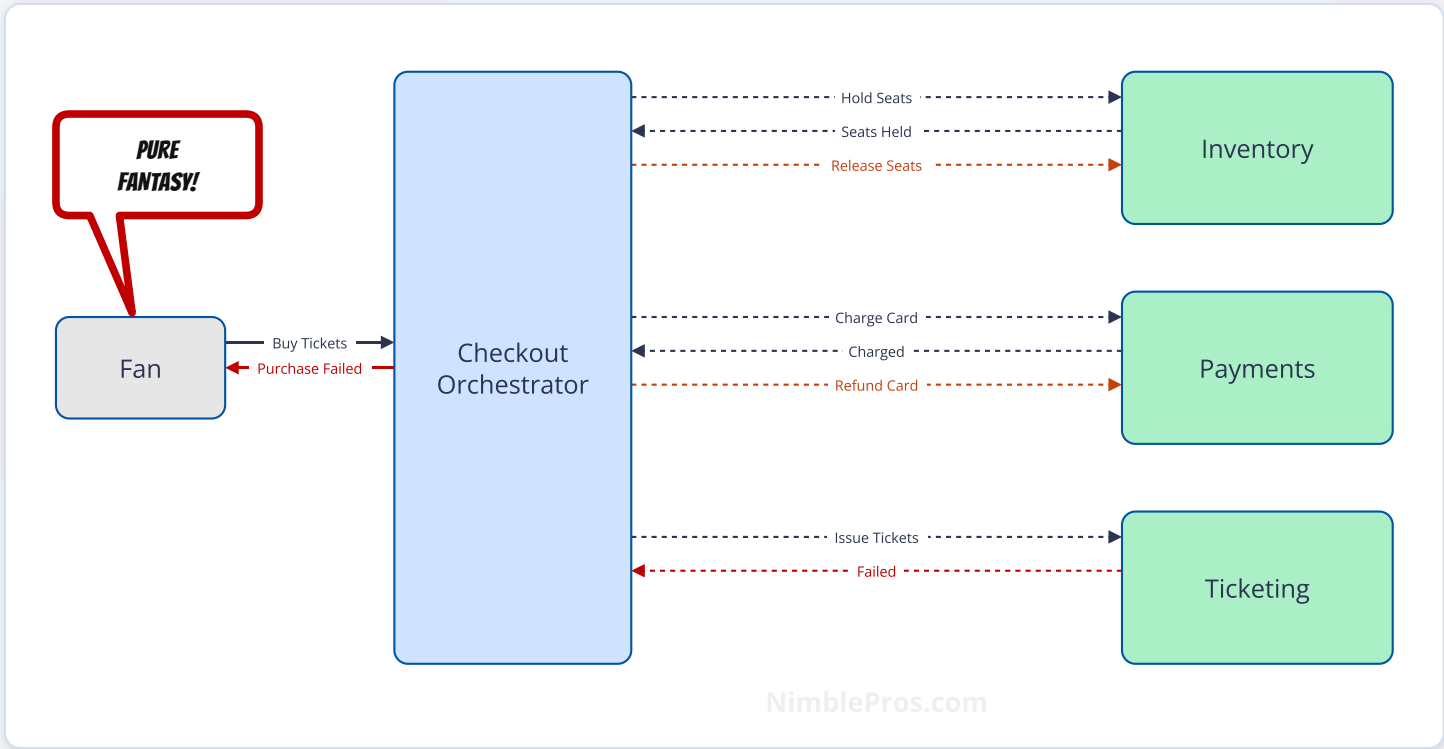
Time Travel Saga in action



Fantasy Fiction Saga

- **Asynchronous, atomic, orchestrated**
- The orchestrator sends messages to every service and doesn’t wait
- On failure, it must undo every step, including replies still in flight
- The fan still waits; usually a sign to change one of the choices
- Sample: `samples/fantasy-fiction-saga`

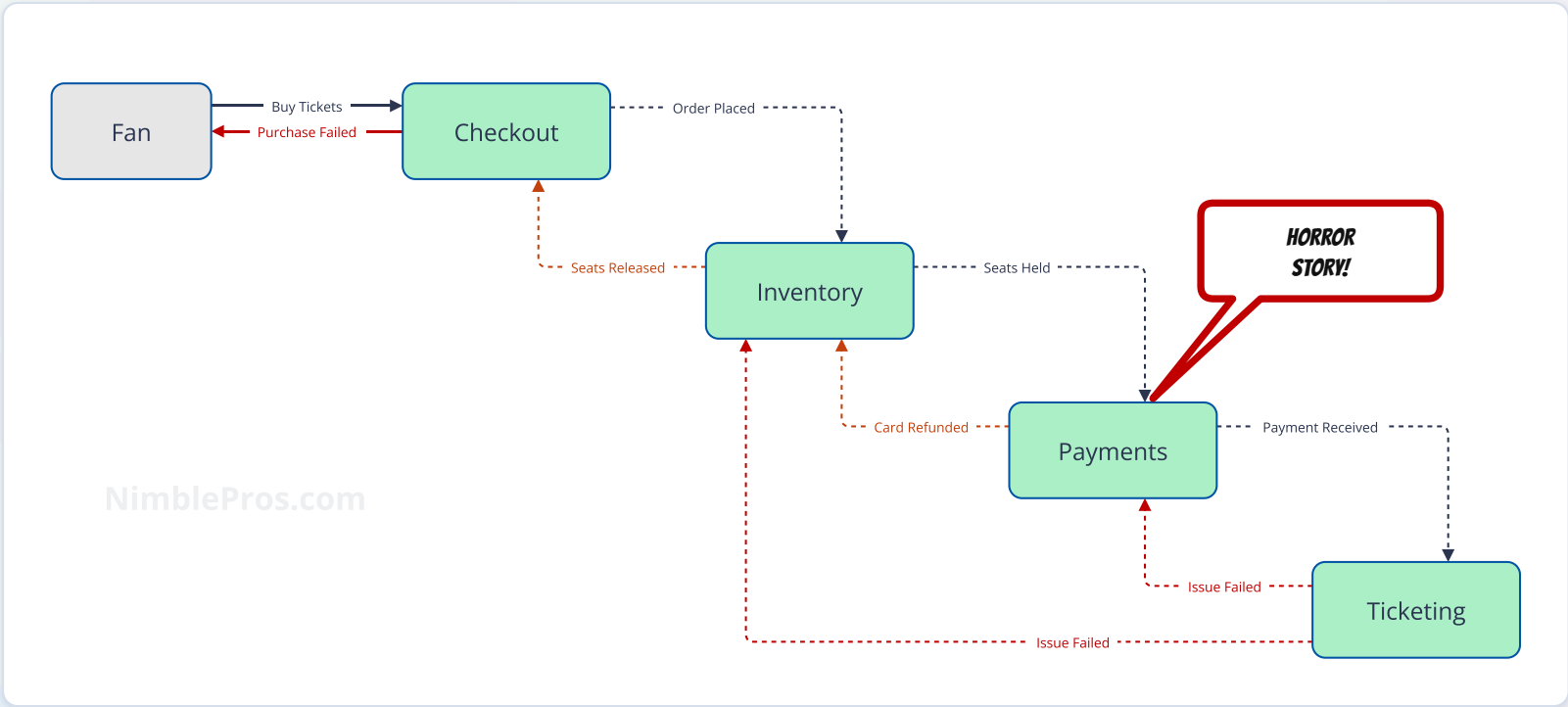
Fantasy Fiction Saga in action



Horror Story Saga

- **Asynchronous, atomic, choreographed**
- Services react to each other’s messages; no one coordinates
- On failure, every upstream service must hear about it, and compensations race
- The loosest coordination paired with the strictest consistency
- Sample: `samples/horror-story-saga`

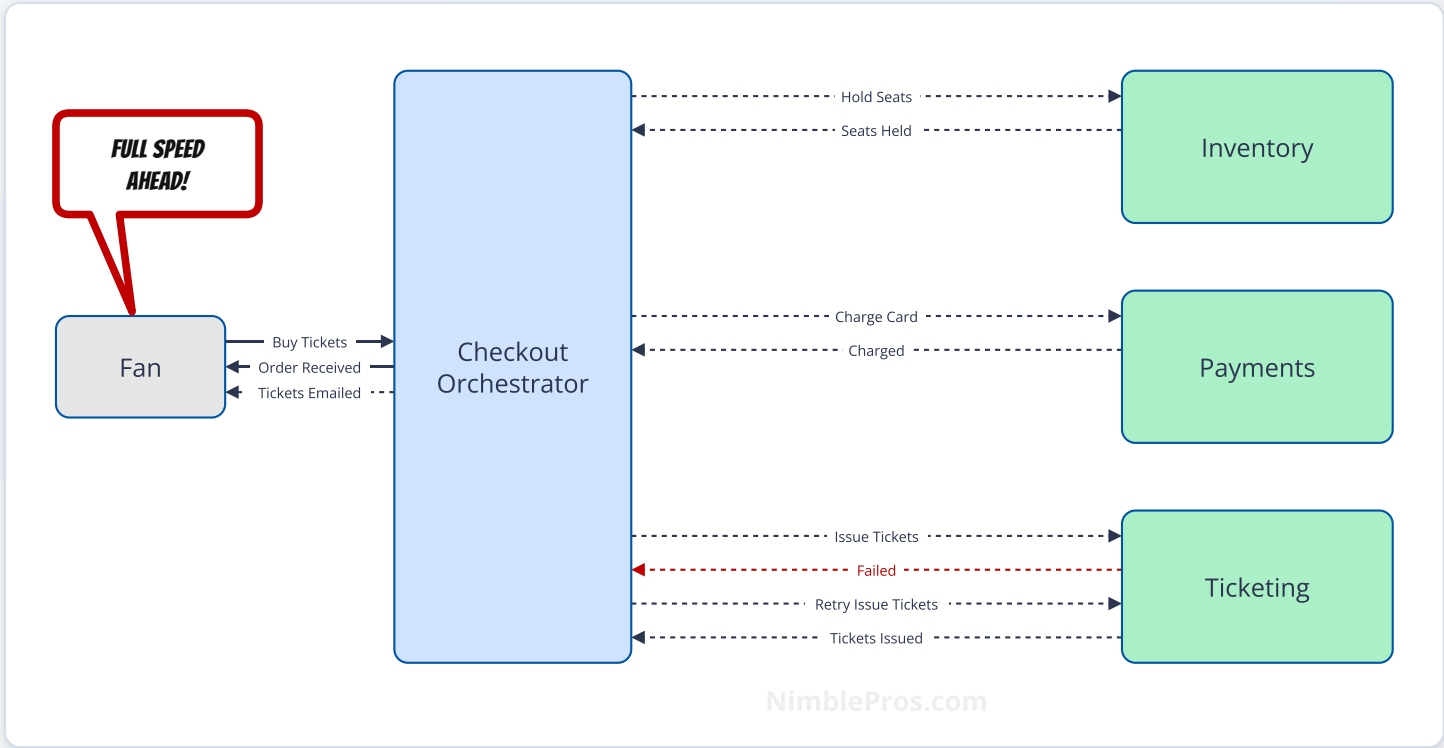
Horror Story Saga in action



Parallel Saga

- **Asynchronous, eventual, orchestrated**
- The orchestrator answers the fan at once, then messages every service in parallel
- On failure, nothing is undone: the orchestrator retries
- High throughput with one owner for errors; users see "pending"
- Sample: `samples/parallel-saga`

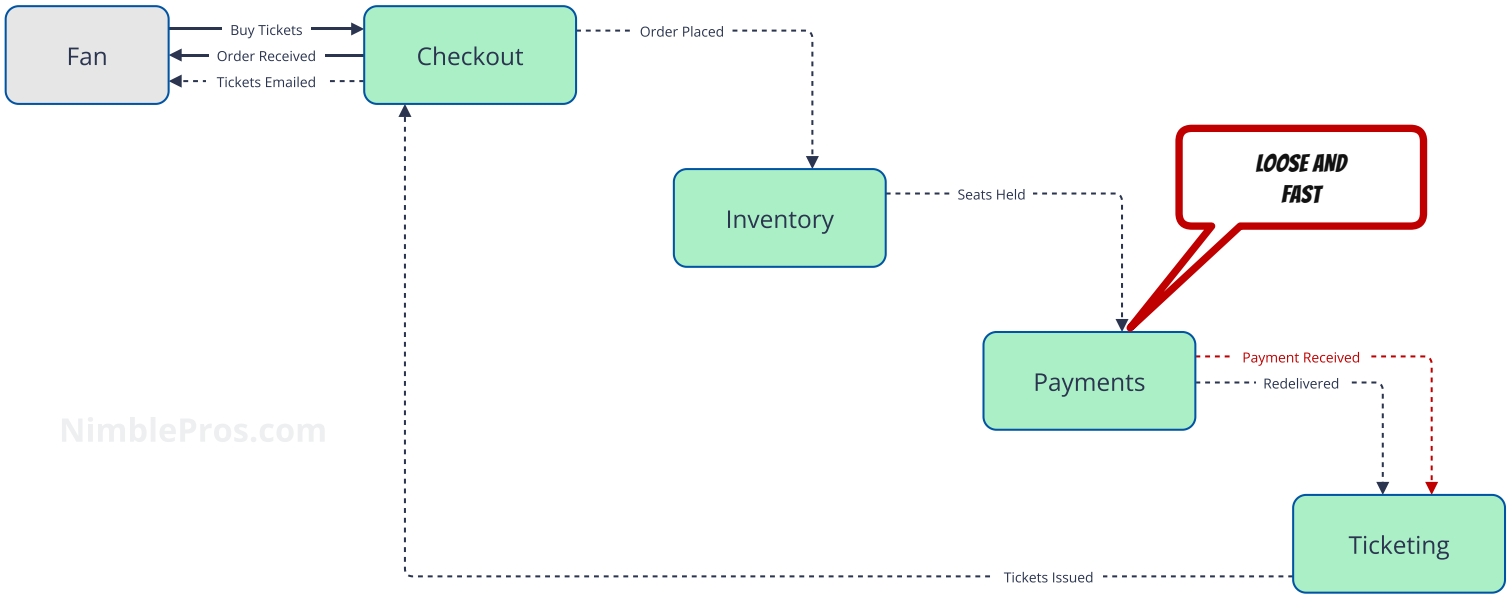
Parallel Saga in action



Anthology Saga

- **Asynchronous, eventual, choreographed**
- Checkout answers the fan at once; events flow from service to service
- On failure, nothing is undone: the message is redelivered
- Lowest coupling, highest scalability; no one owns the workflow
- Sample: `samples/anthology-saga`

Anthology Saga in action



11

SECTION

Wrap-Up

Module 11 · 10 min

Key takeaway

Trade-offs, characteristics, and decisions matter more than any single style.

Next steps

- Explore the samples
- Write ADRs for your own decisions
- Add agents and skills to your repos
- Protect your architecture with fitness functions

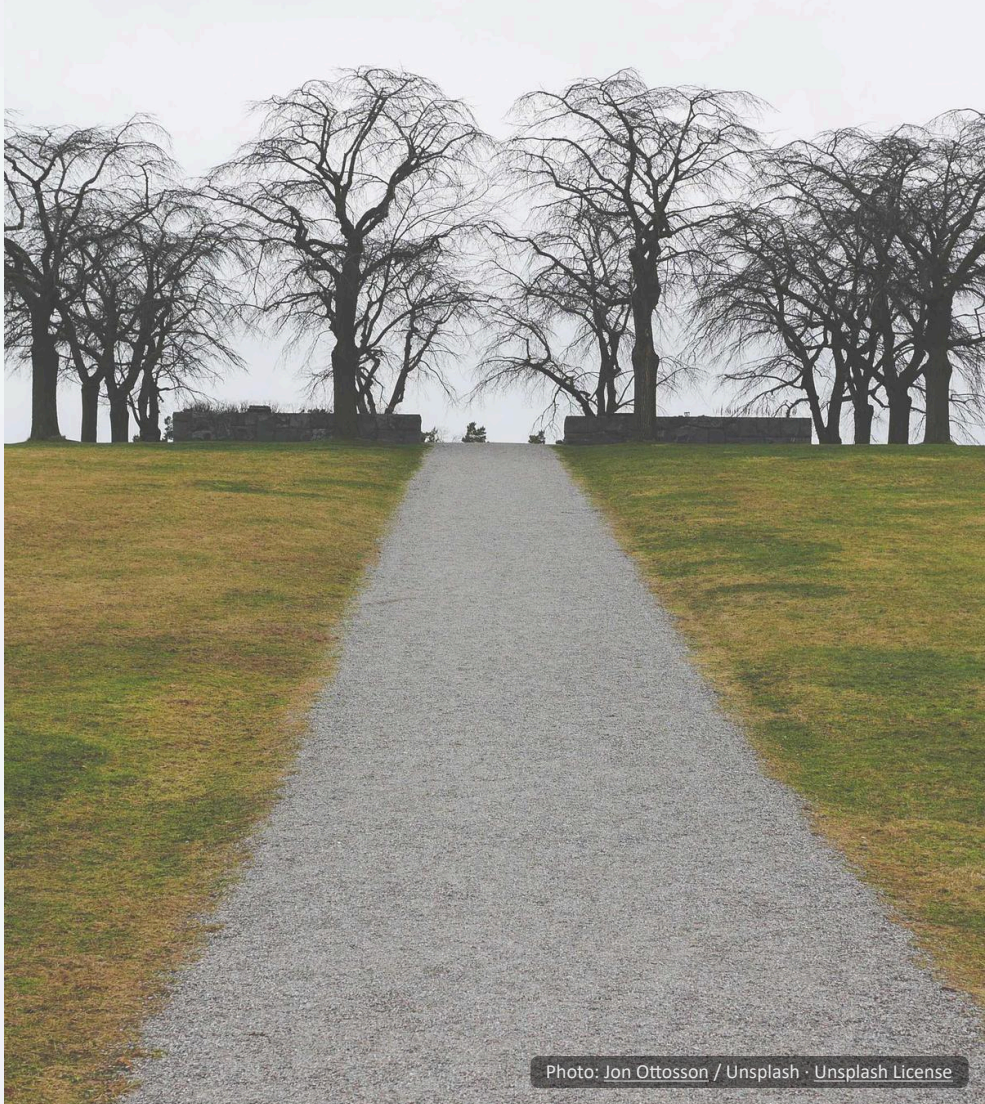


Photo: Jon Ottosson / Unsplash · Unsplash License



NimblePros

TRAINING & MENTORING

LET'S KEEP BUILDING

Thank you

O dear friend,
Wouldst though please complete this short evaluation?

SpreaView

Review my Session

